claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-ae3854c503f476217.jsonl`
- SHA-256: `6672bff103e6c22a79c76585160aae12923540288c0f13284a0388f8152f611e`
- Source modified: `2026-06-10T05:44:02+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T05:33:14.933Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
TASK: deep ENGINEERING + METHODOLOGY audit of the indexer's eval/calibration/learning stack. Read docs/CODE_MAP.md section 2.3 first, then deep-read: backend/eval/ (metrics.py, annotations.py, calibration.py, harness.py, batch.py, regression.py, training.py, classifier.py, calibrate_wasb.py, calibrate_local.py, distill_local.py, export_wasb_segments.py, gemini_refine.py, eval_partial_labels.py, rally_seq_proto.py, audio/e1_probe.py), backend/tools/rally_slicer.py, run_eval.py, run_phase3_eval.py, run_regression.py, run_process_and_stitch.py.
ASSESS: (1) statistical/methodology soundness ? greedy IoU matching, LOVO honesty, partial-label window restriction, overfit guards, the numpy-only LogisticRegression, rally_seq_proto's feature design (incl. the known fps-non-normalization), threshold-transfer problem; (2) is the regression gate (run_regression.py) actually a usable CI gate ? semantics, silent-pass paths, baseline handling; (3) code quality ? duplication of constants (BASELINE/TUNED/paddings), shared de-facto APIs, fragile imports; (4) correctness bugs in metrics/scoring (off-by-one, empty-set handling, division by zero); (5) whether this stack can honestly support the owned-model Phase-0 plan (Gen-0 TAL harness): what is missing or untrustworthy.
Severity-rate honestly. New findings beat re-reporting CODE_MAP gotchas. Quantified claims (e.g. 'LOVO with n=6 means ±X noise') should be reasoned, not hand-waved.

2. user (2026-06-10T05:33:21.653Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-255 of 330 total (27423 tokens, cap 25000). Call Read with offset=256 limit=255 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

````
# CODE_MAP.md

**Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

**Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

**LEGEND**
- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `§` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

## 1. PIPELINE

### 1A. Runtime: video file -> highlight reel
```
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-defaults, never fatal)
                                                  @backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig
       -> video_id = MD5(whole file)              @backend/utils/hashing.py::compute_video_id (single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)
       -> validate @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
            FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/pipeline/validators/__init__.py)
            [validation FAIL -> persist status="failed", return early; report STILL emitted in finally]
       -> db.add_video(video_id, filepath, status, [r.dict()]) @backend/infrastructure/database.py
       -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
       -> seg = segmenter_registry.get(seg_name)(db, global_config)   ? @backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
       -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
            [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]
       -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs ? even on validation failure / exception; NEVER raises; grafts response["reports"])
       -> select segment ids -> [(start,end)] (+ stitching padding)
       -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name) @backend/pipeline/stitcher/compiler.py
            per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4
```
Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):
```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
 -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted via db.add_candidate_segment]
  -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip, prompt)   ? provider_registry
  -> P4 db.add_play_segment + db.link_candidate_to_segment
```
````

```
Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) -> add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
```
WasbHybridSegmenter.process_video @backend/pipeline/segmenters/wasb_hybrid.py
 -> WasbRunner.healthcheck() (gate; not ok -> return []) @backend/pipeline/detectors/wasb_runner.py
 -> WasbRunner.run_predict(video, out_dir)
      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python wasb_infer.py ...'`
      -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
 -> TrackNetRunner.parse_trajectory_csv(csv) @backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
 -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING BRAIN]
```
(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback (30.0, 1280.0).)

### 1C. Calibration / eval flow (NO pipeline run unless told)
```
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage ? {candidates, final}
  -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
  -> harness.evaluate -> EvalReport -> report_to_dict
       -> batch.aggregate (corpus micro/macro)   @backend/eval/batch.py
       -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py [CI gate]
Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate} @backend/eval/calibration.py
Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier) @backend/eval/{calibrate_local,distill_local}.py
Learned segmenter (offline): training.build_training_set(X,y) -> classifier.LogisticRegression.fit/save
Gemini refinement driver (tuning, standalone): gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run @backend/eval/audio/e1_probe.py
```
Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).

---

## 2. SUBSYSTEMS

### 2.0 Orchestration & config
- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
  - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
```

every endpoint NPEs.
83 - `::process_video` `@POST /api/process` -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read).
84 - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.
85 - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
86 - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
87 - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
88 - `@run_eval.py::main` ? score one video's DB preds vs GT CSV. `--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed. `::_default_db_path` resolves via `backend.config.load_config` (`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89 - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig` (same object the live pipeline feeds segmenters).
90 - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare (snapshots even a regressing run); `::_default_db_path` resolves via `backend.config.load_config`.
91
92 - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config import load_config, AppConfig`.
93 - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
94 - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`, `ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching` (`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ? `extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95 - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts** (`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any section (sections discovered dynamically from `model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to `.get`.
96 - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`, `min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`, `chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`, `ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested `tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ? `::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ? a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
97 - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_fps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`); `::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`). `::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`, `conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`). `::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`, `output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`). `::StitchingConfig` (`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal

of 5 sports. `::Config = AppConfig` alias.
98   - `@backend/config/loader.py` ? load/save with defaults-merge.
99   - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root). `::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
100   - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole defaults dict for that section (Pydantic then re-applies field defaults for missing leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).
101   - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.
102   - `@config.json` ? on-disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.
103   - `@setup.py` ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105   ### 2.1 Segmenters (under pipeline/segmenters) ?
106   - `@backend/pipeline/segmenters/base.py`
107   - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
108   - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).
109   - `@backend/pipeline/segmenters/__init__.py` ? imports all 5 (`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.
110   - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config `indexing.wasb.velocity_threshold(0.02)` + shared `dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs `{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit (copy-paste drift).
111   - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig` from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`; `detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
112   - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections). `::_lazy_load_model` (lazy `torchvision` import so module/tests load without it), `::_extract_action_windows_from_chunk`, `::_make_tracker` (per-chunk ByteTrack ? track IDs chunk-local), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU); `yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated aliases still honored.
113   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning table).
114   - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**

(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
115
116   ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117   - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).
118   - `@backend/pipeline/detectors/tracknet_runner.py`
119   - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120   - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled), `::TrajectoryPoint`(frame,visible,x,y).
121   - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `'''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names `<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever). `::_stage_video` "OK" sentinel.
122   - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner) (+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `_wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC for future swappability.
123   - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights` compared raw), `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array), `::_build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW), `::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `--fresh` ignores cache.
124   - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure, no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'), `::count_net_crossings` (sign changes vs median net; deadband jitter zone), `::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds). `::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is `(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local / distill_local / export_wasb_segments.
125
126   ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127   - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`, `::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN) for mean_iou/boundary errors.
128   - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS), `::write_scaffold`.
129   - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. § `PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap >~0.1` = overfit alarm.
130   - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`, `::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over `trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts, make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,

export_wasb_segments, gemini_refine, audio/e1_probe).
131   - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing + net-crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn` (adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a labels file yields no rallies), `::run`/`::main`. § `LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`; manifest JSON shared with distill_local.
132   - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison + optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`,`::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice` (lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
133   - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
134   - `@backend/eval/harness.py` ? DB-pred scorer (no re-run). `::STAGES=('candidates','final')`, `::get_predictions` (candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown stage raises ValueError; reused by regression.py), `::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for batch.aggregate), `::format_report_text`.
135   - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES` (5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default 0.0), `::label_candidates` (same IoU `match_intervals` as evaluator), `::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
136   - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn). `fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default; stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid` clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
137   - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels** (PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ? `window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ? P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate + metrics.
138   - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter = owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean, spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO. `::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments` (smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage** (Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/ calibrate_local.
139   - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court isolation** (spatial gate to the prominent court, needs real court-region detection). Academy = multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140   - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT: duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal), `::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`), `::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-one-video-out needs ?2 videos; final model trained on all.
141   - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT padded clips (precision lever; eval/tuning, NOT the live pipeline). `::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?

FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.
142   - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
`backend.annotations` NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
143
144   ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145   - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k·std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted
(PR #35).
146   - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
147
148   ### 2.4 Stitcher / tools / utils
149   - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.
150   - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp`.
151   - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
152   - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s;
? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
153
154   ### 2.5 Validators ?
155   - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ?
registration order = exec order; ? no dedup), `::registry` (singleton). § `validate(video_path,
config, context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps
each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
156   - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
157   - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
PRODUCER of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `'mp4'` (so MOV-family passes); external `ffprobe`.
158   - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
159   - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES

(too short).
160   - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps
at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail
threshold.
161   - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold`(default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?
magic absolute, not resolution-normalized.
162   - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`;
3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport
-> empty cfg, falls back to defaults, does NOT fail.
163
164   ### 2.6 Sports / Providers / Annotations ?
165   - `@backend/sports/base.py::SportProfile` ABC
(`name`,`get_prompt`,`get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `.name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. § prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt
actively defends against MM:SS; `get()` returns a fresh instance each call.
166   - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(+`::ProviderRegistry`, auto-registers `'gemini'`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. § `analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible
if process dies mid-run.
167   - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
`::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off
`provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` ->
LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`),
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. § `fetch ->
List[Interval]` sorted; contract says RAISE (not `[]`) on unavailable.
168   - `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the
resolved CSV path; `_resolve` -> `<csv_dir>/<video_ref>.csv`; ->
`backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
169   - `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; GS-4 stub;
injectable `::Labeler`, in-process `_jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only
warns, does not block; default `_not_configured` RAISES NotImplementedError). ? get via
`annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as `"auto"` only
because no-arg ctor succeeds.
170   - `@backend/annotations/__init__.py` ? ? `noqa F401` imports of
file_provider/automated_provider are load-bearing (removing de-registers providers).
171
172   ### 2.7 Infrastructure (DB) ?
173   - `@backend/infrastructure/database.py::Database` ? SQLite, 4 tables
(`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in
`::_init_db` (currently =1, `version<1` creates `candidate_segments`). `::_get_connection` (?
re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL silently skipped).
Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_segment`,`link_candidate_to_s
egment`,`query_candidate_segments(detector=,only_unlabeled=)`,`query_play_segments`,`get_video`,
`clear_video_data`. ? candidate_segments is the detector-agnostic fine-tuning extension point
(common fields columns, detector-specifics in `stats`/`detection_params` JSON; index
`idx_candidates_video_detector`). ? `add_video` is `INSERT OR REPLACE` -> cascade-DELETES
dependent segments on re-ingest; events column is `type` not `event_type`; `query_play_segments`
filters use truthiness so `duration_min=0`/empty-string = "no filter"; write helpers swallow
exceptions.
174
175   ### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
176   - `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path,

config, val_results, val_context=None, segmenter_requested=, segmenter_actual=, was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed config (`config.model_dump(...)` if `hasattr model_dump`); short-circuits to None if unavailable/disabled; pulls play_segments/candidates from `db` (read-only `query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to `harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s `finally:`); writes `<video>.report.json`/`.report.md`/`.summary.md` next to video. `::OBSREPORT_AVAILABLE` ? soft-dep gate (True only if `import obsreport` AND `from . import harvest` both succeed). `::report_enabled(config)` reads `config["report"]["enabled"]` default True.
177   - `@backend/reporting/harvest.py` ? records raw FACTS into `obsreport.RunRecorder` (the footgun RULES live in spec.yaml, not here). `::record_run(...)` ? top-level assembler (does NOT write); builds RunRecorder, reads `config_default = config["indexing"]["default_segmenter"]`, runs 4 stage builders + `_highlights`, `rec.finalize()`. `::_AI_SEGMENTERS={yolo_hybrid,tracknet_hybrid,wasb_hybrid,gemini}` ? (add new AI segmenters here). `::video_meta_from_context` (derives VideoMeta from `val_context['ffprobe_meta']`; now also `audio_present` for Input Quality reports extension; § `fps_source ? {unknown,probed,fallback}`), `::_validation_stage`, `::_segmentation_stage` (§ `details.{segmenter_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_requested,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"), `::_ai_stage` (§ `details.{ai_based,provider,model,api_key_present}`; metric `confirmed_segments`), `::_highlights` (coverage_pct). ? `record_run` fps fallback: `fps_source=='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners). `::_SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so pure helpers like video_meta are importable for tests/coverage even without the dep.)
178   - `@backend/reporting/spec.yaml` ? § declarative report layout + footgun engine consumed by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections` (ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each `{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd; `path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule. Rules: `silent_provider_noop`(error: no api key + 0 segs), `ai_empty_vs_no_rallies`(warn), `fps_fallback_used`(warn), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info), `reingest_replaced_prior`(info), `validation_failed`(error: halts indexing).
179
180   ---
181
182   ## 3. DATA CONTRACTS (canonical shapes ? one place)
183
184   § **Trajectory CSV** (produced by tracknet/wasb predict, consumed by `parse_trajectory_csv`):
185   `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel coords.
186   -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
187
188   § **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo `_extract_action_windows_from_chunk`):
189   `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float, mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
190
191   § **candidate_segments DB row** (`db.add_candidate_segment` / `query_candidate_segments`):
192   `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)), stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count}, detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}` (stats/detection_params deserialized on read).
193   `detection_params` keys by detector: wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_threshold,min_action_window_duration}`.
194
195   § **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
196   `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,

receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count, shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare `model_name`; motion adds random `intensity/avg_speed_kmh`.

    § **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time, end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start; `_candidate_id` stamped internally).

    § **GT / golden CSVs** (TWO conventions ? do not conflate):
    - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal or MM:SS/HH:MM:SS; RAISES on bad row.
    - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`, `export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`; SILENTLY skips bad rows.

    § **det_raw.jsonl** (one window/line): `{"w":<window_index>, "f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`); first violation/JSONDecodeError -> truncate-heal.
    § **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in, frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
    § **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv}, ...]}`; ? DB key = file MD5, manifest `video_id` is a label.

    § **eval primitives**: `Interval=Tuple[float,float]` (start<end). `Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP), unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.

    § **WSL detector candidate** (in-WSL, model<->tracker): `{'xy':np.array([x,y]), 'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST rebuild `xy` as np.array (tracker does `np.linalg.norm`).

    § **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}` ? onset-envelope peak.

    § **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages `validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail}; flags `{code,faq_ref}`; `video_meta.fps_source ? {probed,fallback}`. ? stage/metric/detail names are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync: `ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmentation.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinget}`, `validation.status`, `video_meta.fps_source`.

    ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims below are unverified against external source, only the repo-side caller wasb_infer.py was checked)
    - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval', overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
    - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from cfg["model"] at runtime in wasb_infer.run.)
    - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results, hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
    - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()` resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls `build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y` ? consistent.)
    - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,

center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in wasb_infer.run.)
224   - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.
225
226   ---
227
228   ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified here)
229   - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact file used.
230   - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` + `@run_phase3_eval.py`.
231   - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) -> consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` / `@backend/tools/rally_slicer.py`.
232
233   ---
234
235   ## 6. GOTCHAS (cross-cutting footguns)
236   - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single `@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all run_*.py. Manifest `video_id` is a label, NOT the DB key.
237   - **Config access:** `backend/main.py` and all four `run_*.py` load via `backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in `main.py` that only fires if `global_config`/`.indexing` is `None` (the ASGI-import edge case).
238   - **Segmenter default:** single source of truth = model `IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships `gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `'motion'` getattr fallback noted above.
239   - **`AppConfig.get` must return dict sections, not models:** the legacy `.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also silently keeps typo'd config keys (no rejection).
240   - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI `--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the same `global_config.output.output_dir` ? honored end-to-end.
241   - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also returns `[]` on real failure -> empty != "no rallies". The reporting `silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in the report.
242   - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ? absent -> no-op.
243   - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops firing. Add rules in spec.yaml, not code.
244   - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
245   - **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246   - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo modules).
247   - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248   - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in `output/<db_name>` before any eval/regression.
249   - **Validator ordering coupling:** exec order is Format -> ResolutionFPS ->

```
PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless
`format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6
correlation are NOT config-overridable.
250     - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue
`PRAGMA foreign_keys=ON` per connection.
251     - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs
tracknet's `is None` check.
252     - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
253     - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared
API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-
CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by
`distill_local`; `harness.get_predictions` reused by `regression.regress`.
254     - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255     - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
```

3. user (2026-06-10T05:33:23.194Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/:
__init__.py
__pycache__
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
export_wasb_segments.py
gemini_refine.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
training.py

C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/audio/:
__init__.py
__pycache__
e1_probe.py
C:/Users/avidu/Projects/badminton-highlight-indexer/run_all_tests.py
C:/Users/avidu/Projects/badminton-highlight-indexer/run_eval.py
C:/Users/avidu/Projects/badminton-highlight-indexer/run_phase3_eval.py
C:/Users/avidu/Projects/badminton-highlight-indexer/run_process_and_stitch.py
C:/Users/avidu/Projects/badminton-highlight-indexer/run_regression.py
C:/Users/avidu/Projects/badminton-highlight-indexer/setup.py
```

4. assistant (2026-06-10T05:33:27.684Z)

Now let me deep-read the eval stack files.

5. user (2026-06-10T05:33:28.646Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
```

```
    """

    from dataclasses import dataclass, field
    from typing import List, Tuple, Optional

    Interval = Tuple[float, float]


    def interval_iou(a: Interval, b: Interval) -> float:
        """Temporal intersection-over-union of two intervals.

        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
        overlap / union, both measured as durations on the timeline.
        """
        a_start, a_end = a
        b_start, b_end = b
        if a_end <= a_start or b_end <= b_start:
            return 0.0
        inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
        if inter <= 0.0:
            return 0.0
        union = (a_end - a_start) + (b_end - b_start) - inter
        return inter / union if union > 0 else 0.0


    @dataclass
    class Match:
        """One matched predicted?ground-truth pair."""
        pred_index: int
        gt_index: int
        iou: float


    @dataclass
    class MatchResult:
        """Outcome of matching predictions to ground truth at a given IoU threshold."""
        matches: List[Match] = field(default_factory=list)
        unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
        unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)


    def match_intervals(preds: List[Interval], gts: List[Interval],
                        iou_threshold: float = 0.5) -> MatchResult:
        """Greedy one-to-one matching of predictions to ground truth by descending IoU.

        Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
        are claimed highest-IoU first, and each prediction/GT can be used once. This
        keeps a single prediction from being credited for two ground-truth rallies
        (and vice versa).
        """
        candidates = []
        for pi, p in enumerate(preds):
            for gi, g in enumerate(gts):
                iou = interval_iou(p, g)
                if iou >= iou_threshold:
                    candidates.append((iou, pi, gi))
        # Highest IoU first; ties broken by indices for determinism.
        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

        used_pred, used_gt = set(), set()
        matches: List[Match] = []
        for iou, pi, gi in candidates:
            if pi in used_pred or gi in used_gt:
                continue
```

```
            used_pred.add(pi)
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
        false_positives: int
        false_negatives: int
        precision: float
        recall: float
        f1: float
        mean_iou: float            # over matched pairs (0.0 if none)
        mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
        mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds

        def as_dict(self) -> dict:
            return self.__dict__.copy()


    def score(preds: List[Interval], gts: List[Interval],
              iou_threshold: float = 0.5,
              match_result: Optional[MatchResult] = None) -> Score:
        """Compute precision/recall/F1, mean matched IoU, and boundary errors.

        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
        extracting the failure lists); otherwise matching is done here.
        """
        mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)

        tp = len(mr.matches)
        fp = len(mr.unmatched_pred_indices)
        fn = len(mr.unmatched_gt_indices)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0

        if mr.matches:
            mean_iou = sum(m.iou for m in mr.matches) / tp
            mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
            mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
        else:
            mean_iou = mean_start_err = mean_end_err = 0.0

        return Score(
            iou_threshold=iou_threshold,
            num_pred=len(preds), num_gt=len(gts),
            true_positives=tp, false_positives=fp, false_negatives=fn,
            precision=precision, recall=recall, f1=f1,
```

```
            mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
        )


    def pr_curve(preds: List[Interval], gts: List[Interval],
                 thresholds: Optional[List[float]] = None) -> List[Score]:
        """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).

        Lets you see strict-vs-lenient behaviour rather than a single operating point.
        """
        if thresholds is None:
            thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
        return [score(preds, gts, iou_threshold=t) for t in thresholds]

```

6. user (2026-06-10T05:33:29.122Z)

```
    """Load hand-annotated ground-truth rally timestamps from CSV.

    Annotation file format (one file per video):

        start,end
        00:01:12,00:01:39
        102.5,131.0

    - A header row `start,end` is optional (auto-detected and skipped).
    - Each timestamp is either decimal seconds (`102.5`) or colon-separated
      `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
    - Blank lines and `#` comment lines are ignored.
    """

    import csv
    import logging
    from typing import List, Tuple

    logger = logging.getLogger(__name__)

    Interval = Tuple[float, float]


    def parse_timestamp(value: str) -> float:
        """Parse a timestamp string to float seconds.

        Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
        Raises ValueError on anything unparseable so bad annotations fail loudly.
        """
        value = value.strip()
        if not value:
            raise ValueError("empty timestamp")
        if ":" in value:
            parts = value.split(":")
            if len(parts) > 3:
                raise ValueError(f"too many ':' segments in timestamp '{value}'")
            total = 0.0
            for part in parts:                # most-significant first (H, M, S)
                total = total * 60 + float(part)
            return total
        return float(value)


    def load_annotations(csv_path: str) -> List[Interval]:
        """Load and validate ground-truth rally intervals from a CSV file.

        Returns intervals sorted by start time. Raises ValueError on malformed rows
        (bad timestamp, start >= end) so labeling mistakes surface immediately rather
```

```
        than silently skewing scores.
        """
        intervals: List[Interval] = []
        with open(csv_path, newline="") as f:
            reader = csv.reader(f)
            for lineno, row in enumerate(reader, start=1):
                if not row:
                    continue
                first = row[0].strip()
                if not first or first.startswith("#"):
                    continue
                # Skip an optional header row.
                if first.lower() in ("start", "start_time"):
                    continue
                if len(row) < 2:
                    raise ValueError(f"{csv_path}:{lineno}: expected 'start,end', got
{row!r}")
                try:
                    start = parse_timestamp(row[0])
                    end = parse_timestamp(row[1])
                except ValueError as e:
                    raise ValueError(f"{csv_path}:{lineno}: {e}") from e
                if start >= end:
                    raise ValueError(f"{csv_path}:{lineno}: start ({start}) >= end ({end})")
                intervals.append((start, end))

        intervals.sort(key=lambda iv: iv[0])
        logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
        return intervals


    def write_scaffold(csv_path: str) -> None:
        """Write an empty annotation file with just the header, for hand-filling."""
        with open(csv_path, "w", newline="") as f:
            writer = csv.writer(f)
            writer.writerow(["start", "end"])

```

7. user (2026-06-10T05:33:29.422Z)

```
    """Quantify rally-segmentation improvement ? and guard against overfitting.

    This is the measurement layer for per-video / per-corpus calibration. It answers
    two questions honestly:

    1. **"Did precision improve, by how much?"** ? `improvement_report` scores a
       baseline config vs a tuned config on the same ground truth and reports the
       before/after metrics and the % delta.

    2. **"Is that gain real or overfit?"** ? calibration must be measured on data the
       tuning never saw, otherwise you're just memorising. Two protocols:
       - `cross_validate` ? k-fold over one video's rallies: pick the config on the
         train folds, score on the held-out fold. The **generalization_gap**
         (train ? test) is the overfit alarm.
       - `leave_one_video_out` ? the protocol that matters as more videos arrive:
         pick the config on all *other* videos, score on the held-out video. Reports
         per-video held-out scores + mean±std. This is the number to trust.

    Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
    List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
    harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
    """
    from __future__ import annotations

    import statistics as _st
```

```
    from typing import Callable, Dict, List, Sequence, Tuple, Any

    from backend.eval.metrics import score, Score, Interval

    Config = Any                              # opaque to this module (e.g. a thresholds dict)
    PredictFn = Callable[[Config], List[Interval]]


    def pct_improvement(before: float, after: float) -> float:
        """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
        if before <= 0:
            return float("inf") if after > 0 else 0.0
        return (after - before) / before * 100.0


    def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) -> Score:
        """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
        return score(preds, gts, iou_threshold)


    def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
        """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
        if n <= 0:
            return []
        k = max(2, min(k, n))                 # need >=2 folds; cap at n
        folds = [list(range(i, n, k)) for i in range(k)]   # round-robin ? balanced, deterministic
        splits = []
        for f in range(k):
            test = folds[f]
            train = [i for i in range(n) if i not in set(test)]
            if test and train:
                splits.append((train, test))
        return splits


    def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
                    metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
        """Pick the config whose predictions maximise `metric` against `gts`."""
        best_cfg, best_val = None, -1.0
        for cfg in configs:
            val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
            if val > best_val:
                best_cfg, best_val = cfg, val
        return best_cfg, best_val


    def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
                       k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) -> Dict[str, Any]:
        """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).

        Per fold: choose the best config on the train rallies, score it on the held-out
        rallies. A large `generalization_gap` (train >> test) means the thresholds are
        overfitting that video's quirks.

        Default metric is **recall** on purpose: predictions are global (whole-video
        windows), so scoring them against a *fold subset* of GT would unfairly count the
        other rallies' windows as false positives ? precision/F1 are not well-defined
        per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")
        is the honest within-video generalization signal. For precision/F1, use
        `leave_one_video_out` (each video scored whole) or `improvement_report` on a
```

```
        held-out video.
        """
        splits = kfold_indices(len(gts), k)
        if not splits:
            return {"error": "not enough rallies for cross-validation", "n": len(gts)}
        train_scores, test_scores, picked = [], [], []
        for train_idx, test_idx in splits:
            gts_tr = [gts[i] for i in train_idx]
            gts_te = [gts[i] for i in test_idx]
            cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
            te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
            train_scores.append(tr)
            test_scores.append(te)
            picked.append(cfg)
        mean_test = _st.mean(test_scores)
        mean_train = _st.mean(train_scores)
        return {
            "metric": metric,
            "folds": len(splits),
            "mean_test": mean_test,
            "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
            "mean_train": mean_train,
            "generalization_gap": mean_train - mean_test,   # >~0.1 = likely overfitting
            "picked_configs": picked,
        }


    def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
                            metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
Any]:
        """Leave-one-video-out CV ? the honest metric as the corpus grows.

        `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
        For each held-out video: pick the config that's best *pooled across the other
        videos*, then score it on the held-out one. The mean held-out score is the
        "precision on unseen videos" number; per-video spread shows stability.
        """
        if len(videos) < 2:
            return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
        per_video = []
        for i, held in enumerate(videos):
            others = [v for j, v in enumerate(videos) if j != i]

            def pooled_score(cfg: Config) -> float:
                vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
metric)
                        for v in others]
                return _st.mean(vals)

            best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
t: t[1])
            held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
iou_threshold), metric)
            per_video.append({"video": held.get("name", f"video_{i}"),
                              "held_out": held_score, "train_pooled": best_train, "config":
best_cfg})
        held_scores = [p["held_out"] for p in per_video]
        train_scores = [p["train_pooled"] for p in per_video]
        return {
            "metric": metric,
            "mean_held_out": _st.mean(held_scores),
            "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
            "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),
            "per_video": per_video,
        }
```

```
146
147
148     def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
149                            gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
150         """The "+X% precision" headline: baseline vs tuned config on the same GT.
151
152         NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
153         or read the cross-validated `mean_test` from `cross_validate` instead.
154         """
155         b = score(predict_fn(baseline_config), gts, iou_threshold)
156         t = score(predict_fn(tuned_config), gts, iou_threshold)
157         return {
158             "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
159             "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
160             "precision_delta_pts": t.precision - b.precision,
161             "precision_delta_pct": pct_improvement(b.precision, t.precision),
162             "f1_delta_pct": pct_improvement(b.f1, t.f1),
163         }
164
```

8. user (2026-06-10T05:33:34.377Z)

```
1       """Score the pipeline's detected rallies against ground-truth annotations.
2
3       Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4       so scoring is fast, deterministic, and free. Two prediction "stages" can be
5       scored:
6
7         - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
8           the shuttle/person detector found, before AI confirmation.
9         - ``final``      ? confirmed play segments (``play_segments`` table). After the
10          AI handoff trimmed/validated boundaries.
11
12      Comparing the two tells you whether the weak link is *detection* (candidates
13      already miss rallies) or *segmentation* (candidates catch them but final is
14      wrong) ? see docs/EVALUATION.md.
15      """
16
17      import logging
18      from dataclasses import dataclass, field
19      from typing import List, Dict, Optional
20
21      from backend.infrastructure.database import Database
22      from backend.eval import metrics
23      from backend.eval.metrics import Interval, Score, MatchResult
24
25      logger = logging.getLogger(__name__)
26
27      STAGES = ("candidates", "final")
28
29
30      @dataclass
31      class StageReport:
32          stage: str
33          pred_intervals: List[Interval]
34          score: Score
35          match_result: MatchResult
36          pr_curve: List[Score] = field(default_factory=list)
37
38
39      @dataclass
```

```
    class EvalReport:
        video_id: str
        iou_threshold: float
        gt_intervals: List[Interval]
        stages: Dict[str, StageReport] = field(default_factory=dict)


    def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
        """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
        ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
        ivs.sort(key=lambda iv: iv[0])
        return ivs


    def get_predictions(db: Database, video_id: str, stage: str,
                        detector: Optional[str] = None) -> List[Interval]:
        """Fetch predicted rally intervals for one stage from the DB."""
        if stage == "candidates":
            rows = db.query_candidate_segments(video_id, detector=detector)
        elif stage == "final":
            rows = db.query_play_segments(video_id, {})
        else:
            raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
        return _rows_to_intervals(rows)


    def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
                 stages: List[str], iou_threshold: float = 0.5,
                 detector: Optional[str] = None,
                 with_pr_curve: bool = False) -> EvalReport:
        """Score the requested stages for one video against ground truth."""
        report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
                            gt_intervals=gt_intervals)
        for stage in stages:
            preds = get_predictions(db, video_id, stage, detector=detector)
            mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
            s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
            curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
            report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
                                               score=s, match_result=mr, pr_curve=curve)
        return report


    # ---------------------------------------------------------------------- reporting

    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    def report_to_dict(report: EvalReport) -> dict:
        """JSON-serialisable view of an EvalReport."""
        out = {
            "video_id": report.video_id,
            "iou_threshold": report.iou_threshold,
            "num_ground_truth": len(report.gt_intervals),
            "stages": {},
        }
        for stage, sr in report.stages.items():
            out["stages"][stage] = {
                "score": sr.score.as_dict(),
                "pr_curve": [s.as_dict() for s in sr.pr_curve],
            }
```

```
        return out


    def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
        """Human-readable console report for one video."""
        lines = [
            "=" * 64,
            f" Rally evaluation ? video '{report.video_id}'",
            f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
            "=" * 64,
            f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
            f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
            "-" * 64,
        ]
        for stage in report.stages:
            s = report.stages[stage].score
            lines.append(
                f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
                f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
                f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
            )
        lines.append("=" * 64)

        if show_failures:
            for stage, sr in report.stages.items():
                misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
                fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
                lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
                for iv in misses:
                    lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
                lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
                for iv in fps:
                    lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")

        return "\n".join(lines)

```

9. user (2026-06-10T05:33:35.257Z)

```
    """Batch evaluation across a collector export manifest.

    Phase 3 of the collector->indexer bridge: walk the `manifest.json` emitted by
    sports-data-collector's `curate export`, (optionally) process each video through
    a segmenter so detections land in the indexer DB, then score every video's
    detected rallies against its ground-truth CSV and aggregate the results.

    This module holds the *pure* pieces (path resolution, stage selection, metric
    aggregation) so they can be unit-tested without touching video files, the GPU,
    or any API. The orchestration/side-effects live in `run_phase3_eval.py`.
    """

    import os
    from typing import Dict, List, Optional


    # Segmenters that only ever populate the `final` play_segments table (no raw
    # candidate stage), so scoring `candidates` for them would be meaninglessly empty.
    _FINAL_ONLY_SEGMENTERS = {"motion", "gemini"}


```

```
    def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
        """Resolve a manifest `local_path` to an absolute path.

        The collector stores paths like ``./data/videos\\shuttleset_1.mp4`` relative
        to its own repo root and with Windows separators. Normalise separators, strip
        a leading ``./``, and resolve relative paths against `videos_root` (the
        collector root). Returns None for a missing/empty path.
        """
        if not local_path:
            return None
        p = local_path.replace("\\", "/")
        if p.startswith("./"):
            p = p[2:]
        # Cross-platform: treat Windows drive-letter absolute paths (C:/...) as absolute
        # even when running on Linux (os.path.isabs("C:/...") == False on Unix).
        # This keeps collector manifest paths working in tests and mixed environments.
        if (len(p) > 1 and p[1] == ":" and p[0].isalpha()) or os.path.isabs(p):
            return os.path.normpath(p)
        return os.path.normpath(os.path.join(videos_root, p))


    def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
        """Pick which prediction stages to score.

        ``auto`` scores both stages for two-stage detectors (yolo_hybrid/tracknet_*)
        but only ``final`` for segmenters that don't emit candidates. An explicit
        request ('candidates' | 'final' | 'both') overrides the heuristic.
        """
        if requested == "both":
            return ["candidates", "final"]
        if requested in ("candidates", "final"):
            return [requested]
        # auto
        if segmenter in _FINAL_ONLY_SEGMENTERS:
            return ["final"]
        return ["candidates", "final"]


    def _safe_div(num: float, den: float) -> float:
        return num / den if den else 0.0


    def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
        """Aggregate per-video stage scores into corpus-level metrics.

        `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
        ``stages[stage]["score"]`` with tp/fp/fn/mean_iou/...). For each stage we
        compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
        a TP-weighted mean IoU, plus the macro mean of per-video F1.
        """
        out: Dict[str, dict] = {}
        for stage in stages:
            tp = fp = fn = 0
            iou_weighted = 0.0
            f1s: List[float] = []
            n_videos = 0
            for rep in per_video:
                sr = rep.get("stages", {}).get(stage)
                if not sr:
                    continue
                s = sr["score"]
                tp += s["true_positives"]
                fp += s["false_positives"]
                fn += s["false_negatives"]
                iou_weighted += s["mean_iou"] * s["true_positives"]
```

```
                f1s.append(s["f1"])
                n_videos += 1
            precision = _safe_div(tp, tp + fp)
            recall = _safe_div(tp, tp + fn)
            out[stage] = {
                "videos": n_videos,
                "tp": tp, "fp": fp, "fn": fn,
                "micro_precision": precision,
                "micro_recall": recall,
                "micro_f1": _safe_div(2 * precision * recall, precision + recall),
                "macro_f1": _safe_div(sum(f1s), len(f1s)),
                "mean_iou": _safe_div(iou_weighted, tp),
            }
        return out


    def format_aggregate(agg: Dict[str, dict]) -> str:
        """Human-readable corpus-level summary table."""
        lines = [
            "=" * 78,
            " CORPUS AGGREGATE",
            "=" * 78,
            f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
            f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
            "-" * 78,
        ]
        for stage, a in agg.items():
            lines.append(
                f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
                f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
                f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
            )
        lines.append("=" * 78)
        return "\n".join(lines)

```

10. user (2026-06-10T05:33:35.382Z)

```
    """Regression runner ? verify the production tool against golden-set ground truth.

    Implements the explicit ask: *"given a video, get a golden set created and verify
    its precision and recall against the production tool."* This is deliberately a thin
    wrapper over machinery that already exists:

        ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
        predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
        comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
        baseline      ? a per-video snapshot of "known good" numbers [this module]

    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
    """
    import os
    import json
    import logging
    from dataclasses import dataclass, asdict
    from typing import Dict, List, Optional

    from backend.infrastructure.database import Database
    from backend.eval import metrics
    from backend.eval.harness import get_predictions
    from backend.annotations import annotation_registry

```

```
    logger = logging.getLogger(__name__)

    DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
    DEFAULT_TOLERANCE = 0.05


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Use this to (re)baseline after a *legitimate* production improvement, so old
        baselines don't raise false regressions. Stored as plain JSON for easy diffing.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)


    # ---------------------------------------------------------------- the runner

    def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
                stage: str = "final", iou_threshold: float = 0.5,
                detector: Optional[str] = None,
                tolerance: float = DEFAULT_TOLERANCE,
                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
```

```
        """Score stored predictions for one video against ground truth and compare to
baseline.

        `ground_truth` is a list of (start, end) intervals ? typically obtained from an
        AnnotationProvider (see `regress_with_provider`). A regression is flagged when
        precision OR recall falls more than `tolerance` below the snapshotted baseline.
        If no baseline exists yet, `regressed` is False (nothing to compare to) and the
        caller can choose to snapshot the current numbers via `save_baseline`.
        """
        preds = get_predictions(db, video_id, stage, detector=detector)
        s = metrics.score(preds, ground_truth, iou_threshold)

        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
            if s.precision < base_p - tolerance:
                regressed = True
                reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
        )


    def regress_with_provider(db: Database, video_id: str, video_ref: str,
                              tier: str = "file", stage: str = "final",
                              iou_threshold: float = 0.5,
                              detector: Optional[str] = None,
                              tolerance: float = DEFAULT_TOLERANCE,
                              baseline_path: str = DEFAULT_BASELINE_PATH,
                              guideline: Optional[str] = None,
                              sport: str = "badminton",
                              provider_kwargs: Optional[dict] = None) -> RegressionResult:
        """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.

        `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
        once those providers land). `video_ref` is whatever that provider expects (a CSV
        path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
        predictions in the DB (the file MD5).
        """
        provider = annotation_registry.get(tier, **(provider_kwargs or {}))
        gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
        return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)


    def format_result(r: RegressionResult) -> str:
        verdict = "REGRESSED" if r.regressed else "ok"
        head = (f"[{verdict}] {r.video_id} ({r.stage}) "
```

```
153                 f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
154         if r.baseline_precision is not None:
155             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
156         return head + "".join(f"\n   - {why}" for why in r.reasons)
157
```

11. user (2026-06-10T05:33:39.595Z)

```
1       """Build a training set for the offline, API-free learned rally segmenter.
2
3       Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4       a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5       produces candidate rally windows with features; the collector's ground-truth
6       rally intervals tell us which windows are real. A lightweight classifier learns
7       that mapping, replacing the paid Gemini refinement with a local model.
8
9       This module is the *data layer* ? pure and unit-tested:
10
11        - `label_candidates`  ? generic: label windows positive/negative by whether they
12          match a ground-truth rally (same IoU matching the evaluator uses, so training
13          labels are consistent with how we score).
14        - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15          persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16        - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18      No model training or I/O here; that lives in the trainer/segmenter once the
19      substrate is chosen from the Phase 3 candidate-recall result.
20      """
21
22      from typing import Callable, Dict, List, Tuple
23
24      from backend.eval.metrics import Interval, match_intervals
25
26      # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27      # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28      YOLO_FEATURE_NAMES = [
29          "duration", "peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count",
30      ]
31
32
33      def extract_yolo_features(row: Dict) -> List[float]:
34          """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36          `stats` is already JSON-deserialized by the DB layer. Missing fields default
37          to 0.0 so a sparse/old row still yields a well-formed vector.
38          """
39          stats = row.get("stats") or {}
40          duration = float(row["end_time"]) - float(row["start_time"])
41          return [
42              duration,
43              float(stats.get("peak_velocity", 0.0)),
44              float(stats.get("mean_velocity", 0.0)),
45              float(stats.get("active_frame_count", 0)),
46              float(stats.get("track_id_count", 0)),
47          ]
48
49
50      def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51                           iou_threshold: float = 0.5) -> List[int]:
52          """Label each candidate window 1 (real rally) or 0 (spurious).
53
54          A window is positive iff it is matched one-to-one to a ground-truth rally at
55          `iou_threshold` ? the *same* matching the evaluator uses for true positives,
```

```
        so the classifier is trained on exactly what we score. Substrate-agnostic:
        works for YOLO or WASB-derived windows alike.
        """
        mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
        positive = {m.pred_index for m in mr.matches}
        return [1 if i in positive else 0 for i in range(len(candidate_intervals))]


    def build_training_set(
        candidate_rows: List[Dict],
        gt_intervals: List[Interval],
        iou_threshold: float = 0.5,
        feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
    ) -> Tuple[List[List[float]], List[int], List[Interval]]:
        """Turn one video's candidate rows + ground truth into (X, y, intervals).

        `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
        features without changing the labeling logic. Returns the feature matrix `X`,
        binary labels `y`, and the candidate intervals (handy for downstream merging).
        """
        intervals: List[Interval] = [
            (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
        ]
        y = label_candidates(intervals, gt_intervals, iou_threshold)
        X = [feature_fn(r) for r in candidate_rows]
        return X, y, intervals
```

12. user (2026-06-10T05:33:40.140Z)

```
    """A tiny, dependency-free logistic-regression classifier (numpy only).

    The offline learned segmenter (ADR-004) must run locally with no external API and
    no heavy ML stack ? the project already depends on numpy, and sklearn is not
    installed. This implements just enough: standardized features, balanced-class
    gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
    model is a small, human-readable, version-controllable file.

    It deliberately stays simple: the rally-window features are few and roughly
    linearly separable (high player motion + sustained activity => real rally), so a
    well-regularized linear model is a sensible, inspectable baseline. Swap in a
    richer model later behind the same fit/predict_proba/save/load interface.
    """

    import json
    from typing import List, Optional

    import numpy as np


    class LogisticRegression:
        def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
            self.lr = lr
            self.epochs = epochs
            self.l2 = l2
            self.w: Optional[np.ndarray] = None
            self.b: float = 0.0
            self.mean: Optional[np.ndarray] = None
            self.std: Optional[np.ndarray] = None
            self.feature_names: Optional[List[str]] = None

        @staticmethod
        def _sigmoid(z: np.ndarray) -> np.ndarray:
            # Clip for numerical stability (avoid overflow in exp).
            return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
```

```
36
37          def _standardize(self, X: np.ndarray) -> np.ndarray:
38              return (X - self.mean) / self.std
39
40          def fit(self, X, y, feature_names: Optional[List[str]] = None,
41                  class_weight: str = "balanced") -> "LogisticRegression":
42              """Fit with optional balanced class weighting (rally windows are imbalanced
43              ? many spurious candidates per real rally)."""
44              X = np.asarray(X, dtype=float)
45              y = np.asarray(y, dtype=float)
46              if X.ndim != 2 or X.shape[0] == 0:
47                  raise ValueError("X must be a non-empty 2D array")
48              self.feature_names = feature_names
49
50              self.mean = X.mean(axis=0)
51              self.std = X.std(axis=0)
52              self.std[self.std == 0] = 1.0  # guard constant features
53              Xs = self._standardize(X)
54              n, d = Xs.shape
55
56              if class_weight == "balanced":
57                  n_pos = max(1, int(y.sum()))
58                  n_neg = max(1, int((1 - y).sum()))
59                  w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60              else:
61                  w_pos = w_neg = 1.0
62              sw = np.where(y == 1, w_pos, w_neg)
63
64              self.w = np.zeros(d)
65              self.b = 0.0
66              for _ in range(self.epochs):
67                  p = self._sigmoid(Xs @ self.w + self.b)
68                  err = (p - y) * sw
69                  grad_w = Xs.T @ err / n + self.l2 * self.w
70                  grad_b = float(err.mean())
71                  self.w -= self.lr * grad_w
72                  self.b -= self.lr * grad_b
73              return self
74
75          def predict_proba(self, X) -> np.ndarray:
76              if self.w is None:
77                  raise RuntimeError("model is not fitted")
78              Xs = self._standardize(np.asarray(X, dtype=float))
79              return self._sigmoid(Xs @ self.w + self.b)
80
81          def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82              return (self.predict_proba(X) >= threshold).astype(int)
83
84          # --- persistence (small JSON, no pickle/joblib) ---------------------------
85
86          def to_dict(self) -> dict:
87              return {
88                  "model": "logreg",
89                  "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90                  "w": self.w.tolist() if self.w is not None else None,
91                  "b": self.b,
92                  "mean": self.mean.tolist() if self.mean is not None else None,
93                  "std": self.std.tolist() if self.std is not None else None,
94                  "feature_names": self.feature_names,
95              }
96
97          @classmethod
98          def from_dict(cls, d: dict) -> "LogisticRegression":
99              m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100             m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
```

```
            m.b = float(d.get("b", 0.0))
            m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
            m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
            m.feature_names = d.get("feature_names")
            return m

        def save(self, path: str) -> None:
            with open(path, "w") as f:
                json.dump(self.to_dict(), f, indent=2)

        @classmethod
        def load(cls, path: str) -> "LogisticRegression":
            with open(path) as f:
                return cls.from_dict(json.load(f))

```

13. user (2026-06-10T05:33:45.835Z)

```
      """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".

      Wires the already-tested pieces together:
        WASB trajectory CSV (Frame,Visibility,X,Y)
          -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
          -> backend.eval.calibration (improvement_report / cross_validate)

      Run (after a full-video WASB pass exists):
          python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
              --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920

      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
      same GT is an **optimistic** upper bound. The trustworthy number is the
      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
      `leave_one_video_out` for precision/F1 generalization).
      """
      from __future__ import annotations

      import csv
      import argparse
      from typing import List, Tuple, Callable

      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
      from backend.eval.calibration import (
          select_best, improvement_report, cross_validate, evaluate,
      )

      Interval = Tuple[float, float]
      Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)

      BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults


      def load_golden_gts(csv_path: str) -> List[Interval]:
          """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
          gts: List[Interval] = []
          with open(csv_path, newline="", encoding="utf-8") as f:
              reader = csv.DictReader(f)
              reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
              for row in reader:
                  row = {(k or "").strip().lower(): v for k, v in row.items()}
                  try:
```

```
                    s = float((row.get("start_time") or "").strip())
                    e = float((row.get("end_time") or "").strip())
                except (TypeError, ValueError):
                    continue
                if e > s:
                    gts.append((s, e))
        return sorted(gts)


    def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
        """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: Config) -> List[Interval]:
            vt, mg, mwd = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width,
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            return [(w["start"], w["end"]) for w in wins]

        return predict_fn


    def default_grid() -> List[Config]:
        return [(vt, mg, mwd)
                for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
                for mg in (1.0, 2.0, 3.0)
                for mwd in (1.0, 2.0, 3.0)]


    def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
            iou: float = 0.5) -> dict:
        gts = load_golden_gts(labels_csv)
        if not gts:
            raise SystemExit(f"no usable golden rallies in {labels_csv}")
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        grid = default_grid()

        base = evaluate(predict_fn(BASELINE), gts, iou)
        best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
        fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)   # OPTIMISTIC
(fit on full GT)
        cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall

        print("=== WASB rally-segmentation calibration ===")
        print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
        print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
              f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
        print(f"tuned cfg    {best_cfg}: f1={best_f1:.3f}")
        print(f"  precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
              f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
        print(f"  f1        {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
              f"({fit['f1_delta_pct']:+.1f}%)")
        print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
              f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
              f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
        print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
        return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
```

```
101
102
103     def main():
104         ap = argparse.ArgumentParser()
105         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
106         ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
107         ap.add_argument("--fps", type=float, default=59.94)
108         ap.add_argument("--frame-width", type=float, default=1920.0)
109         ap.add_argument("--iou", type=float, default=0.5)
110         args = ap.parse_args()
111         run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
112
113
114     if __name__ == "__main__":
115         main()
116
```

14. user (2026-06-10T05:33:46.401Z)

```
1       """Distill Gemini (teacher) into the LOCAL-ONLY pipeline (student) ? calibrate the
2       on-device WASB+heuristic knobs to match Gemini's rally labels, so production runs
3       100% local (no cloud, privacy lever intact, ~zero marginal cost).
4
5       Pipeline being tuned (all local, no Gemini at runtime):
6           WASB trajectory CSV -> trajectory_to_action_windows(velocity, merge_gap, min_dur)
7                                -> rally_gate (min_crossings)            -> rally windows
8
9       Ground truth = Gemini full-context silver labels (one CSV per video). With >=2
10      videos we use leave-one-video-out (tune on the others, score the held-out one) ?
11      the honest "how precise is the local pipeline on an unseen video" number. The
12      output is a single local config + the precision it buys, with NO Gemini dependency
13      shipped.
14
15      Reuses: TrackNetRunner windowing, rally_gate, calibrate_wasb.load_golden_gts,
16      calibration.{select_best, leave_one_video_out, improvement_report}.
17
18      Run:
19          python -m backend.eval.calibrate_local --manifest videos.json
20      where videos.json = [{"name","trajectory","fps","frame_width","labels"}, ...]
21      (labels = a Gemini full-context CSV; trajectory = the WASB CSV).
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Callable, Dict, List, Tuple
28
29      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30      from backend.pipeline.detectors import rally_gate
31      from backend.eval.calibrate_wasb import load_golden_gts
32      from backend.eval.calibration import select_best, leave_one_video_out,
improvement_report, evaluate
33
34      # (velocity_thresh, merge_gap, min_window_duration, min_crossings)
35      LocalConfig = Tuple[float, float, float, int]
36      BASELINE_LOCAL: LocalConfig = (0.02, 2.0, 2.0, 0)   # today's local default (windowing,
no gate)
37
38
39      def make_local_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[LocalConfig], list]:
40          """predict_fn(cfg) -> rally windows from a cached WASB trajectory, LOCAL ONLY
41          (windowing thresholds + the shuttle net-crossing gate). Parsed once."""
```

```
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        def predict_fn(cfg: LocalConfig) -> List[Tuple[float, float]]:
            vt, mg, mwd, mc = cfg
            wins = TrackNetRunner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width,
                velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
            )
            wins = [(w["start"], w["end"]) for w in wins]
            if mc > 0:
                wins = rally_gate.apply_gate(points, wins, fps, min_crossings=mc)
            return wins

        return predict_fn


    def local_grid() -> List[LocalConfig]:
        return [(vt, mg, mwd, mc)
                for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
                for mg in (1.0, 2.0, 3.0)
                for mwd in (1.0, 2.0, 3.0)
                for mc in (0, 1, 2, 3)]


    def build_videos(specs: List[Dict]) -> List[Dict]:
        videos = []
        for s in specs:
            pf = make_local_predict_fn(s["trajectory"], float(s["fps"]),
float(s["frame_width"]))
            gts = load_golden_gts(s["labels"])
            if not gts:
                raise SystemExit(f"no usable labels in {s['labels']} for {s.get('name')}")
            videos.append({"name": s.get("name", s["trajectory"]), "predict_fn": pf, "gts":
gts})
        return videos


    def run(specs: List[Dict], iou: float = 0.5, metric: str = "f1") -> dict:
        videos = build_videos(specs)
        grid = local_grid()
        print(f"=== Local-only calibration vs Gemini silver-GT ({len(videos)} videos, "
              f"{len(grid)} configs, IoU>={iou}, optimise {metric}) ===")

        # Per-video: today's baseline vs the best-fit local config (optimistic ? fit on that
video).
        per_video = []
        for v in videos:
            best_cfg, best_val = select_best(v["predict_fn"], grid, v["gts"], metric, iou)
            imp = improvement_report(v["predict_fn"], BASELINE_LOCAL, best_cfg, v["gts"],
iou)
            b, t = imp["baseline"], imp["tuned"]
            print(f"\n[{v['name']}] {len(v['gts'])} Gemini rallies")
            print(f"  baseline {BASELINE_LOCAL}: P={b['precision']:.3f} R={b['recall']:.3f}
F1={b['f1']:.3f}")
            print(f"  best-fit {best_cfg}: P={t['precision']:.3f} R={t['recall']:.3f}
F1={t['f1']:.3f} "
                  f"(F1 {imp['f1_delta_pct']:+.0f}%, fit-on-self = optimistic)")
            per_video.append({"name": v["name"], "best_cfg": best_cfg, "improvement": imp})

        # Honest cross-video: pick config on the OTHER videos, score the held-out one.
        lovo = leave_one_video_out(videos, grid, metric, iou)
        # Baseline's held-out score (no tuning) for the honest before/after.
        base_held = [evaluate(v["predict_fn"](BASELINE_LOCAL), v["gts"], iou) for v in
videos]
        base_mean = sum(getattr(s, metric) for s in base_held) / len(base_held)
```

```
100
101         print("\n--- HONEST leave-one-video-out (the number to trust) ---")
102         if "error" in lovo:
103             print(f"  {lovo['error']} (need >=2 videos)")
104         else:
105             print(f"  baseline (untuned) mean held-out {metric}: {base_mean:.3f}")
106             print(f"  calibrated mean held-out {metric}:        {lovo['mean_held_out']:.3f}
"
107                   f"± {lovo['std_held_out']:.3f}  (gap {lovo['generalization_gap']:+.3f})")
108             for p in lovo["per_video"]:
109                 print(f"    held-out {p['video']}: {p['held_out']:.3f}  (cfg picked on
others: {p['config']})")
110         return {"per_video": per_video, "lovo": lovo, "baseline_held_out": base_mean,
"metric": metric}
111
112
113     def main() -> None:
114         ap = argparse.ArgumentParser(description="Calibrate the local-only pipeline against
Gemini silver labels.")
115         ap.add_argument("--manifest", required=True,
116                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]")
117         ap.add_argument("--iou", type=float, default=0.5)
118         ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
119         args = ap.parse_args()
120         with open(args.manifest, encoding="utf-8") as f:
121             specs = json.load(f)
122         run(specs, iou=args.iou, metric=args.metric)
123
124
125     if __name__ == "__main__":
126         main()
127
```

15. user (2026-06-10T05:33:51.961Z)

```
1       """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3       Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4       further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5       using a few resolution/fps-INVARIANT local features, so the model transfers across
6       videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8       Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10        filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11        active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12        -> rally windows.
13
14      Training labels come from Gemini full-context labels via the SAME IoU matching the
15      evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16      other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17      compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18      rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20      Run:
21          python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22      """
23      from __future__ import annotations
24
25      import json
26      import argparse
27      from typing import Dict, List, Tuple
28
29      import numpy as np
```

```
30
31      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
32      from backend.pipeline.detectors import rally_gate
33      from backend.eval.calibrate_wasb import load_golden_gts
34      from backend.eval.training import label_candidates
35      from backend.eval.classifier import LogisticRegression
36      from backend.eval.metrics import score
37      from backend.eval.gemini_refine import merge_overlaps
38
39      Interval = Tuple[float, float]
40
41      FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42      # Recall-oriented proposal: deliberately permissive so real rallies are among the
43      # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44      PROPOSAL = (0.005, 1.0, 0.5)
45      BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's production-ish windowing, no learning
46
47
48      def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
49                           proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
50          """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
51          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
52          vt, mg, mwd = proposal
53          wins = TrackNetRunner.trajectory_to_action_windows(
54              points, fps=fps, frame_width=frame_width,
55              velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
56          intervals = [(w["start"], w["end"]) for w in wins]
57          gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-
crossings
58          X: List[List[float]] = []
59          for w, g in zip(wins, gated):
60              dur = max(1e-6, w["end"] - w["start"])
61              win_frames = max(1.0, dur * fps)
62              X.append([
63                  dur,                                            # rally length (sec) ?
fps-invariant
64                  float(w["peak_velocity"]),                      # already normalized by
frame_width
65                  float(w["mean_velocity"]),
66                  float(w["active_frame_count"]) / win_frames,    # active-shuttle ratio ?
fps-invariant
67                  float(g.crossings),                             # net crossings (count) ?
the rally signal
68              ])
69          return intervals, X
70
71
72      def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
73          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
74          vt, mg, mwd = BASELINE_LOCAL
75          wins = TrackNetRunner.trajectory_to_action_windows(
76              points, fps=fps, frame_width=frame_width,
77              velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
78          return [(w["start"], w["end"]) for w in wins]
79
80
81      def load_video(spec: Dict, iou: float = 0.5) -> Dict:
82          fps, fw = float(spec["fps"]), float(spec["frame_width"])
83          intervals, X = build_candidates(spec["trajectory"], fps, fw)
84          gts = load_golden_gts(spec["labels"])
85          y = label_candidates(intervals, gts, iou)
```

```
        return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X": X,
                "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps, fw)}


    def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals: List[Interval],
                        threshold: float = 0.5) -> List[Interval]:
        if not intervals:
            return []
        proba = model.predict_proba(X)
        pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
        return merge_overlaps(pos, gap_tol=1.0)


    def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5, model_out: str = None) -> dict:
        videos = [load_video(s, iou) for s in specs]
        print(f"=== Distilled local rally classifier vs Gemini silver-GT "
              f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
        for v in videos:
            ceil = score(v["intervals"], v["gts"], iou).recall
            print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
                  f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

        result: dict = {"videos": [v["name"] for v in videos]}
        if len(videos) >= 2:
            print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-out) ---")
            clf, base = [], []
            for i, held in enumerate(videos):
                others = [v for j, v in enumerate(videos) if j != i]
                X = [row for v in others for row in v["X"]]
                y = [t for v in others for t in v["y"]]
                model = LogisticRegression().fit(X, y, FEATURE_NAMES)
                preds = predict_rallies(model, held["X"], held["intervals"], threshold)
                sc = score(preds, held["gts"], iou)
                bsc = score(held["baseline"], held["gts"], iou)
                clf.append(sc)
                base.append(bsc)
                print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f} R={sc.recall:.3f} F1={sc.f1:.3f}"
                      f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f} F1={bsc.f1:.3f}")
            mean = lambda xs, a: sum(getattr(s, a) for s in xs) / len(xs)
            print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f} R={mean(clf,'recall'):.3f} "
                  f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
                  f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
            result["mean_learned_f1"] = mean(clf, "f1")
            result["mean_baseline_f1"] = mean(base, "f1")

        # Final model on ALL videos (for production / running on a fresh held-out video).
        Xall = [row for v in videos for row in v["X"]]
        yall = [t for v in videos for t in v["y"]]
        final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
        weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
        print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
        if model_out:
            final.save(model_out)
            print(f"final model (all videos) -> {model_out}")
            result["model_out"] = model_out
        return result

```

```
143
144     def main() -> None:
145         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
146         ap.add_argument("--manifest", required=True,
147                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
148         ap.add_argument("--iou", type=float, default=0.5)
149         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
150         ap.add_argument("--model-out", default="output/local_rally_model.json")
151         args = ap.parse_args()
152         with open(args.manifest, encoding="utf-8") as f:
153             specs = json.load(f)
154         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
155
156
157     if __name__ == "__main__":
158         main()
159
```

16. user (2026-06-10T05:33:52.565Z)

```
1       """Export tuned WASB rally segments for manual eyeball verification.
2
3       After calibration picks a tuned threshold config, this turns a cached WASB
4       trajectory into the actual rally [start,end] segments that config produces, so the
5       user can answer "are the rallies actually better?" ? by scrubbing the video to each
6       segment, by diffing tuned-vs-golden quantitatively, or by cutting clips.
7
8       Reuses (no new windowing/metrics/cutting logic):
9         - calibrate_wasb.make_predict_fn  -> trajectory -> segments for a given config
10        - eval.metrics.match_intervals/score -> tuned-vs-golden IoU comparison
11        - tools.rally_slicer.slice_rallies   -> optional clip cutting (the exported CSV is
12          written in the golden/slicer schema, so it feeds straight back in)
13
14      Run (uses the tuned config from the first calibration by default):
15          python -m backend.eval.export_wasb_segments --trajectory
~/clips/sample_long_traj.csv \
16              --labels "Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-
width 1920
17          # add --video <mp4> --slice  to also cut one padded clip per tuned rally
18      """
19      from __future__ import annotations
20
21      import csv
22      import os
23      import os.path as osp
24      import argparse
25      from typing import List, Tuple, Dict, Optional
26
27      from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
28      from backend.eval.metrics import match_intervals, score
29      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30      from backend.pipeline.detectors import rally_gate
31
32      Interval = Tuple[float, float]
33      Config = Tuple[float, float, float]
34
35      # The tuned config from the first per-video calibration (velocity, merge_gap, min_dur);
36      # F1 0.588 (baseline) -> 0.746. Override on the CLI to export a different operating
point.
37      TUNED: Config = (0.05, 1.0, 1.0)
38
39      SEG_FIELDS = ["rally_number", "start_time", "end_time", "duration", "start_mmss",
```

```
"end_mmss"]
    COMP_FIELDS = ["type", "ref", "start_time", "end_time", "mmss", "status",
                   "iou", "pred_start", "pred_end", "start_err_s", "end_err_s"]


    def seconds_to_mmss(t: float) -> str:
        """Human-scrubbable timestamp, e.g. 106.031 -> '1:46.031'."""
        t = max(0.0, float(t))
        m = int(t // 60)
        return f"{m}:{t - m * 60:06.3f}"


    def segments_to_rows(segments: List[Interval]) -> List[Dict[str, str]]:
        """Number segments 1..N in time order; emit the golden/slicer CSV schema."""
        rows = []
        for i, (s, e) in enumerate(segments, start=1):
            rows.append({
                "rally_number": str(i),
                "start_time": f"{s:.3f}",
                "end_time": f"{e:.3f}",
                "duration": f"{e - s:.3f}",
                "start_mmss": seconds_to_mmss(s),
                "end_mmss": seconds_to_mmss(e),
            })
        return rows


    def _ensure_parent(path: str) -> None:
        os.makedirs(osp.dirname(osp.abspath(path)) or ".", exist_ok=True)


    def write_segments_csv(path: str, segments: List[Interval]) -> str:
        """Write segments in the golden schema (rally_number/start_time/end_time + mm:ss).

        The schema is what tools.rally_slicer.load_rally_labels consumes, so the file
        doubles as input to clip cutting.
        """
        _ensure_parent(path)
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=SEG_FIELDS)
            w.writeheader()
            w.writerows(segments_to_rows(segments))
        return path


    def build_comparison(preds: List[Interval], gts: List[Interval],
                         iou_threshold: float = 0.5) -> dict:
        """Golden-centric comparison: per golden rally, did a tuned segment match (IoU),
        plus the boundary errors and any extra (false-positive) tuned segments."""
        mr = match_intervals(preds, gts, iou_threshold)
        sc = score(preds, gts, iou_threshold, match_result=mr)
        by_gt = {m.gt_index: m for m in mr.matches}

        rows: List[Dict[str, str]] = []
        for gi, (gs, ge) in enumerate(gts, start=0):
            m = by_gt.get(gi)
            if m is not None:
                ps, pe = preds[m.pred_index]
                rows.append({
                    "type": "golden", "ref": str(gi + 1),
                    "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
                    "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
                    "status": "matched", "iou": f"{m.iou:.3f}",
                    "pred_start": f"{ps:.3f}", "pred_end": f"{pe:.3f}",
                    "start_err_s": f"{ps - gs:+.3f}", "end_err_s": f"{pe - ge:+.3f}",
```

```
                })
            else:
                rows.append({
                    "type": "golden", "ref": str(gi + 1),
                    "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
                    "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
                    "status": "MISS", "iou": "0.000",
                    "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
                })
        for pi in mr.unmatched_pred_indices:                # extra tuned segments = false
positives
            ps, pe = preds[pi]
            rows.append({
                "type": "pred_extra", "ref": str(pi + 1),
                "start_time": f"{ps:.3f}", "end_time": f"{pe:.3f}",
                "mmss": f"{seconds_to_mmss(ps)}-{seconds_to_mmss(pe)}",
                "status": "false_positive", "iou": "",
                "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
            })
        return {"score": sc, "rows": rows}


    def write_comparison_csv(path: str, comparison: dict) -> str:
        _ensure_parent(path)
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
            w.writeheader()
            w.writerows(comparison["rows"])
        return path


    def _with_suffix(path: str, suffix: str) -> str:
        root, ext = osp.splitext(path)
        return root + suffix + ext


    def run(trajectory_csv: str, fps: float, frame_width: float, cfg: Config = TUNED,
            out_csv: str = "output/wasb_tuned_segments.csv", labels_csv: Optional[str] =
None,
            iou: float = 0.5, baseline: bool = False,
            comparison_out: Optional[str] = None, min_crossings: int = 0) -> dict:
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        # Gate A: drop between-point false fires (single service-feed tosses) by requiring
        # the shuttle to cross the net >= min_crossings times within the window.
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv) if min_crossings > 0
else None

        def _gate(segs: List[Interval], label: str) -> List[Interval]:
            if not points:
                return segs
            kept = rally_gate.apply_gate(points, segs, fps, min_crossings=min_crossings)
            print(f"[gate] {label}: {len(segs)} -> {len(kept)} segments "
                  f"(removed {len(segs) - len(kept)} with <{min_crossings} net-crossings)")
            return kept

        tuned_segs = _gate(predict_fn(cfg), "tuned")
        write_segments_csv(out_csv, tuned_segs)
        total = sum(e - s for s, e in tuned_segs)
        print("=== WASB tuned-segment export ===")
        print(f"tuned cfg {cfg}: {len(tuned_segs)} rally segments, {total:.1f}s of play ->
{out_csv}")
        result: dict = {"tuned_cfg": cfg, "num_tuned": len(tuned_segs), "out_csv": out_csv}

        if baseline:
            base_segs = _gate(predict_fn(BASELINE), "baseline")
```

```
            base_out = _with_suffix(out_csv, "_baseline")
            write_segments_csv(base_out, base_segs)
            print(f"baseline cfg {BASELINE}: {len(base_segs)} segments -> {base_out}")
            result["num_baseline"] = len(base_segs)
            result["baseline_csv"] = base_out

        if labels_csv:
            gts = load_golden_gts(labels_csv)
            if not gts:
                raise SystemExit(f"no usable golden rallies in {labels_csv}")
            comp = build_comparison(tuned_segs, gts, iou)
            sc = comp["score"]
            comp_out = comparison_out or _with_suffix(out_csv, "_vs_golden")
            write_comparison_csv(comp_out, comp)
            print(f"\n--- tuned vs golden ({len(gts)} rallies, IoU>={iou}) ---")
            print(f"matched (TP) {sc.true_positives} | misses (FN) {sc.false_negatives} | "
                  f"extra (FP) {sc.false_positives}")
            print(f"precision {sc.precision:.3f} | recall {sc.recall:.3f} | f1 {sc.f1:.3f} |
"
                  f"meanIoU {sc.mean_iou:.3f}")
            print(f"mean boundary error: start {sc.mean_start_error:.2f}s | end
{sc.mean_end_error:.2f}s")
            print(f"per-rally detail -> {comp_out}")
            result["comparison"] = sc.as_dict()
            result["comparison_csv"] = comp_out
        return result


    def _maybe_slice(video: str, segments_csv: str, clips_dir: str) -> None:
        """Cut one padded clip per exported rally (reuses the rally slicer)."""
        from backend.tools.rally_slicer import slice_rallies
        written = slice_rallies(video, segments_csv, ["all"], clips_dir)
        print(f"\ncut {len(written)} clip(s) -> {clips_dir}")


    def main() -> None:
        ap = argparse.ArgumentParser(description="Export tuned WASB rally segments for
eyeballing.")
        ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
        ap.add_argument("--fps", type=float, default=59.94)
        ap.add_argument("--frame-width", type=float, default=1920.0)
        ap.add_argument("--velocity-thresh", type=float, default=TUNED[0])
        ap.add_argument("--merge-gap", type=float, default=TUNED[1])
        ap.add_argument("--min-duration", type=float, default=TUNED[2])
        ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden
comparison")
        ap.add_argument("--iou", type=float, default=0.5)
        ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
        ap.add_argument("--comparison-out", default=None)
        ap.add_argument("--baseline", action="store_true", help="also export baseline-config
segments")
        ap.add_argument("--min-crossings", type=int, default=0,
                        help="gate A: drop windows whose shuttle crosses the net < N times "
                             "(kills single-toss service-feed false fires; 0=off, 2
recommended)")
        ap.add_argument("--video", default=None, help="source video (with --slice) to cut
clips from")
        ap.add_argument("--slice", action="store_true", help="cut one padded clip per tuned
rally")
        ap.add_argument("--clips-dir", default="output/wasb_tuned_clips")
        args = ap.parse_args()

        cfg = (args.velocity_thresh, args.merge_gap, args.min_duration)
        res = run(args.trajectory, args.fps, args.frame_width, cfg=cfg, out_csv=args.out,
```

```
                  labels_csv=args.labels, iou=args.iou, baseline=args.baseline,
                  comparison_out=args.comparison_out, min_crossings=args.min_crossings)
        if args.slice:
            if not args.video:
                raise SystemExit("--slice requires --video")
            _maybe_slice(args.video, res["out_csv"], args.clips_dir)


    if __name__ == "__main__":
        main()

```

17. user (2026-06-10T05:33:58.312Z)

```
      """Evaluate the rally pipeline against PARTIAL human golden labels.

      The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
      ending_reason,sport). While a video is only partially tagged, the labels cover a
      PREFIX of the video ? every rally up to some point is marked, and everything after is
      simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
      the detector's *correct* rallies in the un-labeled tail would be counted as false
      positives and crush precision.

      This harness fixes that by restricting the evaluation to the LABELED WINDOW
      [window_start, window_end] (window_end defaults to the last labeled rally's end) and
      dropping predicted segments that fall outside it. Within that window the human labels
      ARE complete, so precision/recall/F1 are honest.

      It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
        - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
        - pipeline.detectors.rally_gate.apply_gate   (net-crossing "gate A")
        - eval.metrics.match_intervals / score
        - export_wasb_segments.build_comparison / write_comparison_csv  (per-rally detail CSV)

      Run:
          python -m backend.eval.eval_partial_labels \
              --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
              --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
Avi.rallies.csv" \
              --fps 59.94 --frame-width 1920
      """
      from __future__ import annotations

      import argparse
      import os.path as osp
      from typing import List, Optional, Tuple

      from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
      from backend.eval.metrics import score
      from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
      from backend.pipeline.detectors import rally_gate

      Interval = Tuple[float, float]
      Config = Tuple[float, float, float]

      # Operating points to compare. The tuned config wins on its fit-video but over-segments
      # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
      TUNED: Config = (0.05, 1.0, 1.0)


      def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
          """Keep intervals that overlap the labeled window [w0, w1].

```

```
        A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
        in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
        they are neither rewarded nor penalized. GTs are assumed already inside the window.
        """
        return [(s, e) for (s, e) in intervals if e > w0 and s < w1]


    def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
                    window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
        """Score one (config, gate) operating point inside the labeled window."""
        preds = predict_fn(cfg)
        if gate_crossings > 0 and points:
            preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
        preds = restrict_to_window(preds, *window)
        sc = score(preds, gts, iou)
        return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
"preds": preds}


    def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
            window_start: float = 0.0, window_end: Optional[float] = None,
            gate_crossings: int = 2, iou: float = 0.5,
            comparison_out: Optional[str] = None) -> dict:
        gts_all = load_golden_gts(labels_csv)
        if not gts_all:
            raise SystemExit(f"no usable human rallies in {labels_csv}")
        if window_end is None:
            window_end = max(e for _, e in gts_all)
        window = (window_start, window_end)
        gts = restrict_to_window(gts_all, *window)

        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        traj_end = (max(p.frame for p in points) / fps) if points else 0.0

        print("=== Pipeline vs PARTIAL human labels ===")
        print(f"labels: {labels_csv}")
        print(f"  {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
{window_end:.1f}s] "
              f"-> {len(gts)} rallies in-window")
        print(f"trajectory covers ~0-{traj_end:.1f}s "
              f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
than window'})")
        print(f"IoU>={iou}; net-crossing gate K={gate_crossings}\n")

        # Compare the standard operating points: baseline / baseline+gate / tuned /
tuned+gate.
        points_for_gate = points
        candidates = [
            ("baseline", BASELINE, 0),
            (f"baseline+gate{gate_crossings}", BASELINE, gate_crossings),
            ("tuned", TUNED, 0),
            (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
        ]
        results = []
        header = f"{'operating point':<18} {'pred':>4} {'TP':>3} {'FN':>3} {'FP':>3} " \
                 f"{'prec':>6} {'recall':>6} {'F1':>6} {'mIoU':>6} {'sErr':>6} {'eErr':>6}"
        print(header)
        print("-" * len(header))
        for name, cfg, gate in candidates:
            r = _eval_point(predict_fn, cfg, points_for_gate, fps, gts, window, gate, iou)
            sc = r["score"]
            r["name"] = name
            results.append(r)
            print(f"{name:<18} {r['n_pred']:>4} {sc.true_positives:>3}
```

```
{sc.false_negatives:>3} "
                  f"{sc.false_positives:>3} {sc.precision:>6.3f} {sc.recall:>6.3f}
{sc.f1:>6.3f} "
                  f"{sc.mean_iou:>6.3f} {sc.mean_start_error:>6.2f}
{sc.mean_end_error:>6.2f}")

        best = max(results, key=lambda r: r["score"].f1)
        print(f"\nbest operating point by F1: {best['name']} (F1 {best['score'].f1:.3f})")

        # Per-rally detail for the best point: which human rallies matched/missed + extra
fires.
        comp = build_comparison(best["preds"], gts, iou)
        comp_out = comparison_out or osp.join("output", "adarsh_avi_vs_human.csv")
        write_comparison_csv(comp_out, comp)
        print(f"per-rally detail ({best['name']}) -> {comp_out}")

        return {
            "window": window, "n_human_in_window": len(gts),
            "results": [{"name": r["name"], "cfg": r["cfg"], "gate": r["gate"],
                         "n_pred": r["n_pred"], **r["score"].as_dict()} for r in results],
            "best": best["name"], "comparison_csv": comp_out,
        }


    # Grid centered on the two levers the human-GT failure analysis flagged:
    # min_window_duration DOWN (recover short 3-5s rallies) and merge_gap UP (heal
    # fragmented long rallies). Gate is swept on/off because it can over-prune.
    DEFAULT_GRID = [
        (vt, mg, mwd)
        for vt in (0.01, 0.02, 0.03, 0.05)
        for mg in (1.0, 2.0, 3.0, 4.0)
        for mwd in (0.5, 1.0, 1.5, 2.0)
    ]


    def sweep(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
              window_start: float = 0.0, window_end: Optional[float] = None,
              gate_options: Tuple[int, ...] = (0, 2), iou: float = 0.5,
              grid: Optional[List[Config]] = None, top: int = 12) -> dict:
        """Grid-search (velocity, merge_gap, min_dur) x gate vs human labels, in-window.

        WARNING: this is fit-on-self on ONE video -> optimistic. It shows the achievable
        ceiling + which levers matter, NOT a transferable config. The honest operating
        point comes from leave-one-video-out once >=3 videos are labeled.
        """
        grid = grid or DEFAULT_GRID
        gts_all = load_golden_gts(labels_csv)
        if not gts_all:
            raise SystemExit(f"no usable human rallies in {labels_csv}")
        if window_end is None:
            window_end = max(e for _, e in gts_all)
        window = (window_start, window_end)
        gts = restrict_to_window(gts_all, *window)
        predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

        rows: List[dict] = []
        for cfg in grid:
            for gate in gate_options:
                r = _eval_point(predict_fn, cfg, points, fps, gts, window, gate, iou)
                sc = r["score"]
                rows.append({"cfg": cfg, "gate": gate, "n_pred": r["n_pred"], "f1": sc.f1,
                             "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
                             "tp": sc.true_positives, "fp": sc.false_positives, "fn":
```

```
sc.false_negatives})
171         rows.sort(key=lambda x: (x["f1"], x["mean_iou"]), reverse=True)
172
173         print(f"=== Config sweep vs human labels ({len(gts)} rallies in "
174               f"[{window_start:.0f},{window_end:.0f}]s; {len(grid)}x{len(gate_options)}
configs) ===")
175         print("NOTE: fit-on-self on ONE video = optimistic; real test is leave-one-video-
out.\n")
176         hdr = f"{'#':>3} {'veloc':>6} {'merge':>6} {'minDur':>6} {'gate':>4} " \
177               f"{'F1':>6} {'prec':>6} {'recall':>6} {'mIoU':>6} {'TP':>3} {'FP':>3}
{'FN':>3}"
178         print(hdr); print("-" * len(hdr))
179         for i, r in enumerate(rows[:top], 1):
180             vt, mg, mwd = r["cfg"]
181             print(f"{i:>3} {vt:>6.3f} {mg:>6.1f} {mwd:>6.1f} {r['gate']:>4} "
182                   f"{r['f1']:>6.3f} {r['precision']:>6.3f} {r['recall']:>6.3f}
{r['mean_iou']:>6.3f} "
183                   f"{r['tp']:>3} {r['fp']:>3} {r['fn']:>3}")
184         b = rows[0]
185         print(f"\nbest: velocity={b['cfg'][0]} merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]}
"
186               f"gate={b['gate']} -> F1 {b['f1']:.3f} (P {b['precision']:.3f} / R
{b['recall']:.3f})")
187         return {"window": window, "n_gt": len(gts), "ranked": rows, "best": b}
188
189
190     def main() -> None:
191         ap = argparse.ArgumentParser(description=__doc__)
192         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
193         ap.add_argument("--labels", required=True, help="human golden CSV
(rally_number,start_time,end_time,...)")
194         ap.add_argument("--fps", type=float, default=59.94)
195         ap.add_argument("--frame-width", type=float, default=1920.0)
196         ap.add_argument("--window-start", type=float, default=0.0)
197         ap.add_argument("--window-end", type=float, default=None,
198                         help="end of the labeled region (default: last labeled rally end)")
199         ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
recommended)")
200         ap.add_argument("--iou", type=float, default=0.5)
201         ap.add_argument("--comparison-out", default=None)
202         ap.add_argument("--sweep", action="store_true",
203                         help="grid-search configs vs labels (fit-on-self; finds the ceiling
+ best levers)")
204         ap.add_argument("--top", type=int, default=12, help="how many sweep rows to print")
205         args = ap.parse_args()
206         if args.sweep:
207             sweep(args.trajectory, args.labels, args.fps, args.frame_width,
208                   window_start=args.window_start, window_end=args.window_end,
209                   iou=args.iou, top=args.top)
210         else:
211             run(args.trajectory, args.labels, args.fps, args.frame_width,
212                 window_start=args.window_start, window_end=args.window_end,
213                 gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
214
215
216     if __name__ == "__main__":
217         main()
218
```

18. user (2026-06-10T05:34:02.621Z)

```
1       """PROTOTYPE: a learned per-FRAME rally segmenter (vs the unsupervised windowing
heuristic).
2
```

```
    Motivation (2026-06-08 finding): WASB tracks the shuttle well on Kushagra (97% of GT
    rallies contain >=2 net-crossings) yet NO velocity-windowing config segments it
    (per-video F1 ceiling 0.163), while it works on Adarsh (0.73). The rally SIGNAL is in
    the trajectory; the hand-tuned heuristic just can't extract it across footage. This
    prototype tests the hypothesis that a LEARNED model on per-frame trajectory features
    recovers the rally segments ? including on the footage the heuristic fails on.

    Approach (deliberately small, dependency-light: numpy + scipy + the repo's pure-python
    LogisticRegression ? no torch/sklearn):
      1. Per-frame features over a local window from the WASB trajectory: visibility rate,
         mean/max shuttle speed (frame-width-normalized), net-crossing RATE, x/y spread.
         All are scale/camera-robust RATES (unlike a raw velocity threshold).
      2. Per-frame label = inside a human GT rally or not.
      3. LEAVE-ONE-VIDEO-OUT: train the classifier on the other match(es), test on the
         held-out one (the honest cross-footage number; no within-video leakage).
      4. Smooth probabilities -> threshold (tuned on TRAIN) -> contiguous in-rally segments
         -> merge tiny gaps / drop short -> score vs GT at IoU>=0.5 (same metric as the
heuristic).

    This is a directional PROOF-OF-SIGNAL, not a production model (n=2 videos, linear model,
    hand features). It exists to answer one question: is the rally boundary learnable from
    the trajectory where the heuristic fails? Compare its held-out F1 to the heuristic LOVO.

    Run:
        python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
    """
    from __future__ import annotations

    import json
    import argparse
    from typing import Dict, List, Tuple

    import numpy as np
    from scipy.ndimage import uniform_filter1d, maximum_filter1d

    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.eval.classifier import LogisticRegression
    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.metrics import score

    FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
    Interval = Tuple[float, float]


    def _spread(a: np.ndarray) -> float:
        return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0


    def build_frame_arrays(points, frame_width: float) -> Dict:
        """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events."""
        pts = sorted(points, key=lambda p: p.frame)
        N = (pts[-1].frame + 1) if pts else 1
        vis = np.zeros(N); x = np.zeros(N); y = np.zeros(N)
        for p in pts:
            if 0 <= p.frame < N and p.visible and p.x >= 0:
                vis[p.frame] = 1.0; x[p.frame] = float(p.x); y[p.frame] = float(p.y)

        mask = vis > 0
        play_is_x = _spread(x[mask]) >= _spread(y[mask])
        coord = x if play_is_x else y
        net = float(np.median(coord[mask])) if mask.any() else 0.0
        deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)

        speed = np.zeros(N); spd_mask = np.zeros(N); cross = np.zeros(N)
        prev = None
```

```
        for p in pts:
            f = p.frame
            if not (0 <= f < N and vis[f]):
                continue
            if prev is not None:
                df = f - prev
                if df > 0:
                    speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df)
                    spd_mask[f] = 1.0
                    c, pc = coord[f] - net, coord[prev] - net
                    if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
                        cross[f] = 1.0
            prev = f
        return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)


    def features(arr: Dict, fps: float, win_sec: float = 0.75, stride_sec: float = 0.1):
        """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
        N = arr["N"]; fw = arr["fw"]
        size = max(3, int(win_sec * fps) * 2 + 1)
        vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))

        n = uniform_filter1d(vis, size, mode="constant")                      # mean
visibility (== vis_rate)
        sm = uniform_filter1d(spd_mask, size, mode="constant")
        spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
        spd_max = maximum_filter1d(speed, size, mode="constant")
        cross_rate = uniform_filter1d(cross, size, mode="constant") * fps       # crossings
/ second
        xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
        x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
        x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
        ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
        y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
        y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw

        idx = np.arange(0, N, max(1, int(stride_sec * fps)))
        X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
        return np.nan_to_num(X), idx / fps


    def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
        y = np.zeros(len(times))
        for i, t in enumerate(times):
            for s, e in gts:
                if s <= t <= e:
                    y[i] = 1.0
                    break
        return y


    def roc_auc(y: np.ndarray, p: np.ndarray) -> float:
        """Rank-based AUC (Mann-Whitney); pure numpy."""
        pos, neg = p[y > 0.5], p[y <= 0.5]
        if pos.size == 0 or neg.size == 0:
            return float("nan")
        order = np.argsort(p, kind="mergesort")
        ranks = np.empty(len(p)); ranks[order] = np.arange(1, len(p) + 1)
        return float((ranks[y > 0.5].sum() - pos.size * (pos.size + 1) / 2) / (pos.size *
neg.size))
```

```
124
125
126     def probs_to_segments(probs: np.ndarray, times: np.ndarray, threshold: float,
127                           merge_gap: float = 1.0, min_dur: float = 1.0, smooth: int = 7) ->
List[Interval]:
128         p = uniform_filter1d(probs, max(1, smooth), mode="nearest")
129         on = p >= threshold
130         runs: List[Interval] = []
131         i, n = 0, len(on)
132         while i < n:
133             if on[i]:
134                 j = i
135                 while j + 1 < n and on[j + 1]:
136                     j += 1
137                 runs.append((float(times[i]), float(times[j])))
138                 i = j + 1
139             else:
140                 i += 1
141         merged: List[Interval] = []
142         for s, e in runs:
143             if merged and s - merged[-1][1] <= merge_gap:
144                 merged[-1] = (merged[-1][0], e)
145             else:
146                 merged.append((s, e))
147         return [(s, e) for s, e in merged if e - s >= min_dur]
148
149
150     def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
151         f1s = []
152         for v in vids:
153             segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
154             f1s.append(score(segs, v["gts"], 0.5).f1)
155         return sum(f1s) / max(len(f1s), 1)
156
157
158     def run(specs: List[Dict]) -> dict:
159         data = {}
160         for s in specs:
161             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
162             arr = build_frame_arrays(pts, float(s["frame_width"]))
163             X, times = features(arr, float(s["fps"]))
164             gts = load_golden_gts(s["labels"])
165             data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
166         names = list(data)
167         print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
168         print(f"features: {FEAT_NAMES}\n")
169
170         results = []
171         for test in names:
172             train = [data[n] for n in names if n != test]
173             Xtr = np.vstack([v["X"] for v in train]); ytr = np.concatenate([v["y"] for v in
train])
174             clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
175             clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
176             # tune the decision threshold on TRAIN segment-F1 (no test leakage)
177             thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
178             v = data[test]
179             probs = clf.predict_proba(v["X"])
180             auc = roc_auc(v["y"], probs)
181             segs = probs_to_segments(probs, v["times"], float(thr))
182             sc = score(segs, v["gts"], 0.5)
183             # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
```

```
ceiling
184             # from the threshold-transfer gap. Optimistic ? for diagnosis only.
185             othr = max(np.arange(0.2, 0.81, 0.05),
186                        key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
187             osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
188             results.append({"video": test, "thr": float(thr), "auc": auc,
189                             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
190                             "n_pred": len(segs), "n_gt": len(v["gts"]),
191                             "oracle_thr": float(othr), "oracle_f1": osc.f1})
192             print(f"[held-out {test}]  per-frame AUC {auc:.3f}  |  LOVO-threshold F1
{sc.f1:.3f} "
193                   f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train)  |  "
194                   f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
195         mean_f1 = sum(r["f1"] for r in results) / len(results)
196         print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
197         return {"per_video": results, "mean_f1": mean_f1}
198
199
200     def main() -> None:
201         ap = argparse.ArgumentParser(description=__doc__)
202         ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
203         args = ap.parse_args()
204         with open(args.manifest, encoding="utf-8") as f:
205             run(json.load(f))
206
207
208     if __name__ == "__main__":
209         main()
210
```

19. user (2026-06-10T05:34:07.038Z)

```
1       """Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).
2
3       WASB (shuttle-only) over-fires on between-point service feeds. This drives the
4       EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
5       each candidate window is cut to a small padded clip and handed to Gemini, which ?
6       seeing the actual footage ? returns the true rally inside it (tight boundaries) or
7       nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
8       not GB-scale chunks; high-precision because Gemini judges each candidate visually.
9
10      Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
11      sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,
12      calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.
13
14      This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
15      trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.
16
17      Run (GEMINI_API_KEY in env):
18          python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv
\
19              --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
20      """
21      from __future__ import annotations
22
23      import os
24      import os.path as osp
25      import time
26      import argparse
27      import concurrent.futures
28      from typing import List, Tuple, Optional
29
```

```
    from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
    from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry

    Interval = Tuple[float, float]


    def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
        """Union of overlapping intervals, optionally stitching across a small gap.

        gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
brief
        shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
gaps
        (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
logic,
        which over-merged multiple real rallies WASB had bundled into one candidate.)"""
        merged: List[Interval] = []
        for s, e in sorted(intervals):
            if merged and s <= merged[-1][1] + gap_tol:
                merged[-1] = (merged[-1][0], max(merged[-1][1], e))
            else:
                merged.append((s, e))
        return merged


    def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
        """Map a Gemini clip's returned segments back to full-video seconds, clamped to
clip."""
        out: List[Interval] = []
        for sg in raw or []:
            try:
                st = float(sg["start_time"]) + clip_start
                en = float(sg["end_time"]) + clip_start
            except (TypeError, ValueError, KeyError):
                continue
            st = max(clip_start, st)
            en = min(clip_end, en)
            if en > st:
                out.append((round(st, 3), round(en, 3)))
        return out


    def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,
                      pad: float, duration: float, tmpdir: str, idx: int,
                      clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval,
List[Interval]]:
        # Own provider/client per call: the genai client isn't reliably thread-safe, and a
        # shared one across workers throws socket errors. A fresh lazy client is cheap.
        provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
        s, e = window
        cs = max(0.0, s - pad)
        ce = min(duration, e + pad) if duration > 0 else e + pad
        clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
        # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
        # 500MB upload cap even for 4K/5K sources.
        if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True,
scale_width=clip_scale_width):
            return idx, window, []
        try:
            prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
            raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
label=f"cand{idx}")
```

```
            mapped = _offset_segments(raw, cs, ce)
        finally:
            try:
                os.remove(clip)
            except OSError:
                pass
        # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
        # (merge_gap bridged the between-point gaps), and collapsing them to one envelope
swept
        # dead time into a "rally" (precision loss). Tiny invisibility splits are healed
later
        # by the gap-tolerant global merge in run().
        return idx, window, mapped


    def run(video: str, trajectory: str, fps: float, frame_width: float,
            cfg=BASELINE, pad: float = 3.0, model: str = "gemini-flash-latest",
            max_workers: int = 4, out_csv: str = "output/gemini_refined.csv",
            min_dur: float = 2.0, limit: int = 0, clip_scale_width: Optional[int] = 1280) ->
dict:
        api_key = os.environ.get("GEMINI_API_KEY")
        if not api_key:
            raise SystemExit("GEMINI_API_KEY not set in env")
        sport_profile = sport_registry.get("badminton")
        duration = get_video_duration(video)

        candidates = make_predict_fn(trajectory, fps, frame_width)(cfg)
        if limit:
            candidates = candidates[:limit]
        tmpdir = osp.join("output", "gemini_refine_clips")
        os.makedirs(tmpdir, exist_ok=True)

        print(f"[gemini_refine] {len(candidates)} WASB candidates (cfg {cfg}) -> Gemini
{model}, "
              f"pad {pad}s, {max_workers} workers", flush=True)
        t0 = time.time()
        per_candidate = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
            futs = [ex.submit(refine_window, api_key, model, sport_profile, video, w, pad,
duration,
                              tmpdir, i, clip_scale_width)
                    for i, w in enumerate(candidates)]
            for fut in concurrent.futures.as_completed(futs):
                idx, window, mapped = fut.result()
                per_candidate.append((idx, window, mapped))
                tag = "REJECT (no rally)" if not mapped else \
                      " ".join(f"{seconds_to_mmss(a)}-{seconds_to_mmss(b)}" for a, b in
mapped)
                print(f"  cand {idx:>2}
{seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)

        raw_intervals = [iv for _, _, m in per_candidate for iv in m]
        refined = [(s, e) for s, e in merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >=
min_dur]
        write_segments_csv(out_csv, refined)
        try:
            os.rmdir(tmpdir)
        except OSError:
            pass

        kept_cands = sum(1 for _, _, m in per_candidate if m)
        play = sum(e - s for s, e in refined)
        print(f"\n[gemini_refine] {kept_cands}/{len(candidates)} candidates had a rally; "
              f"{len(refined)} merged rallies, {play:.1f}s play -> {out_csv} "
              f"({time.time() - t0:.0f}s)", flush=True)
```

```
        return {"refined": refined, "num_candidates": len(candidates),
                "candidates_with_rally": kept_cands, "out_csv": out_csv}


    def full_video_rallies(video: str, model: str = "gemini-flash-latest",
                           out_csv: str = "output/gemini_fullvideo.csv", clip_scale_width:
Optional[int] = 1280,
                           min_dur: float = 2.0, chunk_sec: float = 120.0, overlap: float =
10.0,
                           max_workers: int = 3) -> List[Interval]:
        """Full-context pass: hand Gemini the WHOLE clip (downscaled, chunked to stay under
the
        upload cap), so it finds every rally itself ? not bounded by WASB's candidate
recall.
        Independent of the trajectory; complements refine()."""
        api_key = os.environ.get("GEMINI_API_KEY")
        if not api_key:
            raise SystemExit("GEMINI_API_KEY not set in env")
        sport_profile = sport_registry.get("badminton")
        duration = get_video_duration(video)
        starts, t = [], 0.0
        while t < duration:
            starts.append(t)
            t += max(1.0, chunk_sec - overlap)
        tmpdir = osp.join("output", "gemini_full_clips")
        os.makedirs(tmpdir, exist_ok=True)

        def do_chunk(ci: int, cstart: float) -> List[Interval]:
            cend = min(cstart + chunk_sec, duration)
            clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
            if not extract_video_chunk(video, cstart, cend - cstart, clip, reencode=True,
scale_width=clip_scale_width):
                return []
            try:
                provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
                prompt = sport_profile.get_prompt(chunk_duration=cend - cstart)
                raw, _ = provider.analyze_video(clip, prompt, chunk_duration=cend - cstart,
label=f"chunk{ci}")
                return _offset_segments(raw, cstart, cend)
            finally:
                try:
                    os.remove(clip)
                except OSError:
                    pass

        print(f"[gemini_full] {len(starts)} chunk(s) of {video} (<= {chunk_sec}s, width
{clip_scale_width}) "
              f"-> Gemini {model}", flush=True)
        with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
            chunks = [f.result() for f in [ex.submit(do_chunk, i, s) for i, s in
enumerate(starts)]]
        rallies = [(s, e) for s, e in merge_overlaps([iv for c in chunks for iv in c],
gap_tol=1.0) if e - s >= min_dur]
        write_segments_csv(out_csv, rallies)
        try:
            os.rmdir(tmpdir)
        except OSError:
            pass
        play = sum(e - s for s, e in rallies)
        print(f"[gemini_full] {len(rallies)} rallies, {play:.1f}s play -> {out_csv}",
flush=True)
        return rallies


```

```
199     def main() -> None:
200         ap = argparse.ArgumentParser(description="Refine WASB candidates with Gemini.")
201         ap.add_argument("--video", required=True)
202         ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required
unless --full-video)")
203         ap.add_argument("--fps", type=float, default=59.94)
204         ap.add_argument("--frame-width", type=float, default=1920.0)
205         ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
206                         help="WASB candidate config to feed Gemini (baseline = whole
rallies, good recall)")
207         ap.add_argument("--pad", type=float, default=3.0)
208         ap.add_argument("--model", default="gemini-flash-latest")
209         ap.add_argument("--max-workers", type=int, default=4)
210         ap.add_argument("--min-dur", type=float, default=2.0)
211         ap.add_argument("--out", default="output/gemini_refined.csv")
212         ap.add_argument("--limit", type=int, default=0, help="only the first N candidates
(smoke test)")
213         ap.add_argument("--clip-scale-width", type=int, default=1280,
214                         help="downscale uploaded clips to this width (0=keep source res)")
215         ap.add_argument("--full-video", action="store_true",
216                         help="full-context pass: send the whole clip (chunked/downscaled) to
Gemini "
217                              "instead of refining WASB candidates (needs no --trajectory)")
218         args = ap.parse_args()
219         scale = args.clip_scale_width or None
220         if args.full_video:
221             full_video_rallies(args.video, model=args.model, out_csv=args.out,
222                                clip_scale_width=scale, min_dur=args.min_dur,
max_workers=args.max_workers)
223             return
224         if not args.trajectory:
225             raise SystemExit("--trajectory is required unless --full-video")
226         cfg = BASELINE if args.config == "baseline" else TUNED
227         run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
228             model=args.model, max_workers=args.max_workers, out_csv=args.out,
229             min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)
230
231
232     if __name__ == "__main__":
233         main()
234
```

20. user (2026-06-10T05:34:07.227Z)

```
1       """E1 ? audio GO/NO-GO probe (docs/AUDIO_RALLY_DETECTION_PLAN.md).
2
3       Answers, on REAL audio: does the shuttle "thwack" survive well enough that hit
4       peaks (a) cluster inside rallies far more than dead time, (b) recover rallies the
5       WASB windows miss (our recall bottleneck), and (c) form usable audio-only rally
6       proposals? No training, no fusion ? purely diagnostic, so we know whether to build
7       the rest of the audio path.
8
9       Reuses: audio.hit_detector, calibrate_wasb.load_golden_gts/make_predict_fn,
10      eval.metrics. Labels may be human-gold OR Gemini-silver ? silver is fine for the
11      SNR/efficacy question E1 asks.
12      """
13      from __future__ import annotations
14
15      import argparse
16      from typing import List, Tuple
17
18      from backend.pipeline.audio.hit_detector import detect_hits_in_video, HitEvent
19      from backend.eval.calibrate_wasb import load_golden_gts, make_predict_fn, BASELINE
20      from backend.eval.metrics import match_intervals, score
21      from backend.utils.video import get_video_duration
```

```
22
23      Interval = Tuple[float, float]
24
25
26      def cluster_hits(hits: List[HitEvent], gap_sec: float = 2.0, min_hits: int = 2,
27                       pad: float = 0.3) -> List[Interval]:
28          """Group hits whose inter-hit gap <= gap_sec into rally proposals; keep clusters
29          with >= min_hits (a lone hit = a service feed, not a rally)."""
30          ts = sorted(h.t for h in hits)
31          if not ts:
32              return []
33          out: List[Interval] = []
34          start = last = ts[0]
35          cnt = 1
36          for t in ts[1:]:
37              if t - last <= gap_sec:
38                  last, cnt = t, cnt + 1
39              else:
40                  if cnt >= min_hits:
41                      out.append((max(0.0, start - pad), last + pad))
42                  start = last = t
43                  cnt = 1
44          if cnt >= min_hits:
45              out.append((max(0.0, start - pad), last + pad))
46          return out
47
48
49      def _hits_in(hits: List[HitEvent], s: float, e: float) -> int:
50          return sum(1 for h in hits if s <= h.t <= e)
51
52
53      def run(video: str, labels_csv: str, trajectory: str, fps: float, frame_width: float,
54              k: float = 2.5, cluster_gap: float = 2.0, min_hits: int = 2, iou: float = 0.5)
-> dict:
55          hits = detect_hits_in_video(video, k=k)
56          gts = load_golden_gts(labels_csv)
57          wasb = make_predict_fn(trajectory, fps, frame_width)(BASELINE)
58          dur = get_video_duration(video) or (max((e for _, e in gts + wasb), default=0.0))
59
60          print(f"=== E1 audio probe: {video} ===")
61          print(f"  {len(hits)} hits detected (k={k}) over {dur:.0f}s =
{len(hits)/max(1,dur)*60:.1f} hits/min | "
62                f"{len(gts)} labeled rallies, {len(wasb)} WASB baseline windows")
63
64          # (1) Discrimination: hit density inside rallies vs dead time.
65          rally_dur = sum(e - s for s, e in gts)
66          dead_dur = max(1e-6, dur - rally_dur)
67          hits_in = sum(_hits_in(hits, s, e) for s, e in gts)
68          hits_out = len(hits) - hits_in
69          dens_in = hits_in / max(1e-6, rally_dur)
70          dens_out = hits_out / dead_dur
71          ratio = dens_in / dens_out if dens_out > 0 else float("inf")
72          rallies_with_hits = sum(1 for s, e in gts if _hits_in(hits, s, e) >= 2)
73          print(f"  [discrimination] in-rally {dens_in:.2f} hits/s vs dead-time {dens_out:.2f}
hits/s "
74                f"? ratio {ratio:.1f}x | rallies with >=2 hits: {rallies_with_hits}/{len(gts)}
"
75                f"({100*rallies_with_hits/max(1,len(gts)):.0f}%)")
76
77          # (2) Recall recovery: of rallies WASB MISSES, how many have an audio hit-cluster?
78          mr = match_intervals(wasb, gts, iou)
79          missed = [gts[i] for i in mr.unmatched_gt_indices]
80          recoverable = [r for r in missed if _hits_in(hits, *r) >= min_hits]
81          rec_rate = len(recoverable) / len(missed) if missed else 0.0
82          print(f"  [recall recovery] WASB misses {len(missed)}/{len(gts)} rallies; "
```

```
83                f"audio hit-clusters sit in {len(recoverable)} of them ? {100*rec_rate:.0f}%
recoverable")
84
85          # (3) Audio-only rally proposals vs labels.
86          props = cluster_hits(hits, cluster_gap, min_hits)
87          sc = score(props, gts, iou)
88          print(f"  [audio-only proposals] {len(props)} rallies | P={sc.precision:.3f}
R={sc.recall:.3f} "
89                f"F1={sc.f1:.3f} (vs WASB-only F1 {score(wasb, gts, iou).f1:.3f})")
90          return {
91              "video": video, "k": k, "n_hits": len(hits), "duration": dur,
92              "discrimination_ratio": ratio, "rallies_with_hits_pct": 100 * rallies_with_hits
/ max(1, len(gts)),
93              "wasb_missed": len(missed), "recall_recoverable_pct": 100 * rec_rate,
94              "audio_only": sc.as_dict(), "wasb_only_f1": score(wasb, gts, iou).f1,
95          }
96
97
98      def main() -> None:
99          ap = argparse.ArgumentParser(description="E1 audio go/no-go probe.")
100         ap.add_argument("--video", required=True)
101         ap.add_argument("--labels", required=True, help="golden or Gemini-silver rally CSV")
102         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV")
103         ap.add_argument("--fps", type=float, default=59.94)
104         ap.add_argument("--frame-width", type=float, default=1920.0)
105         ap.add_argument("--k", type=float, default=2.5, help="onset sensitivity (lower =
more hits)")
106         ap.add_argument("--cluster-gap", type=float, default=2.0)
107         ap.add_argument("--min-hits", type=int, default=2)
108         ap.add_argument("--iou", type=float, default=0.5)
109         args = ap.parse_args()
110         run(args.video, args.labels, args.trajectory, args.fps, args.frame_width,
111             k=args.k, cluster_gap=args.cluster_gap, min_hits=args.min_hits, iou=args.iou)
112
113
114     if __name__ == "__main__":
115         main()
116
```

21. user (2026-06-10T05:34:11.679Z)

```
1       """Slice individual rally clips out of a (long) annotated video.
2
3       Use cases:
4       - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5         boundaries against the labels.
6       - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7         supervision the segmenter is tuned/adapted on for that specific video.
8
9       Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10      end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11          <out_dir>/<video_stem>__rally_<n>.mp4
12
13      Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14      shows the same lead-in/out the model is trained and evaluated against. The window
15      math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16      `slice_rallies`.
17      """
18      from __future__ import annotations
19
20      import argparse
21      import csv
22      import os
23      import subprocess
24      from dataclasses import dataclass
```

```
    from typing import Dict, List, Optional, Tuple

    from backend.eval.annotations import parse_timestamp

    # Match VideoCompiler defaults so sliced clips frame a rally the same way.
    BEGIN_PADDING = 0.5
    END_PADDING = 1.0


    @dataclass
    class ClipWindow:
        rally: str          # kept as a string to allow inserted ids like "7.5"
        start: float        # padded, clamped
        end: float          # padded, clamped


    def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
        """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
        for key in (decimal_key, mmss_key):
            val = (row.get(key) or "").strip()
            if val:
                try:
                    return parse_timestamp(val)
                except ValueError:
                    continue
        return None


    def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
        """Map rally_number -> (start, end) seconds, skipping rows with no usable times.

        Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
        end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
        """
        labels: Dict[str, Tuple[float, float]] = {}
        with open(csv_path, newline="", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
            for row in reader:
                row = {(k or "").strip().lower(): v for k, v in row.items()}
                rally = (row.get("rally_number") or "").strip()
                if not rally:
                    continue
                start = _read_time(row, "start_time", "start_mmss")
                end = _read_time(row, "end_time", "end_mmss")
                if start is None or end is None or end <= start:
                    continue  # missing/degenerate (e.g. a not-yet-filled MISSED row)
                labels[rally] = (start, end)
        return labels


    def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
                             rally_numbers: List[str],
                             begin_pad: float = BEGIN_PADDING,
                             end_pad: float = END_PADDING,
                             video_duration: Optional[float] = None) -> List[ClipWindow]:
        """Apply padding and clamp to the video, for the requested rallies.

        rally_numbers may be ["all"] to take every labeled rally (in time order).
        Unknown rally ids are skipped (caller can diff to warn).
        """
        if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
            wanted = sorted(labels, key=lambda r: labels[r][0])
        else:
```

```
            wanted = [r for r in rally_numbers if r in labels]

        windows: List[ClipWindow] = []
        for rally in wanted:
            start, end = labels[rally]
            ps = max(0.0, start - begin_pad)
            pe = end + end_pad
            if video_duration is not None and video_duration > 0:
                if ps >= video_duration:
                    continue  # entirely past the end
                pe = min(pe, video_duration)
            if pe <= ps:
                continue
            windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
        return windows


    def _probe_duration(video_path: str) -> Optional[float]:
        try:
            out = subprocess.run(
                ["ffprobe", "-v", "error", "-show_entries", "format=duration",
                 "-of", "default=noprint_wrappers=1:nokey=1", video_path],
                capture_output=True, text=True, check=True,
            ).stdout.strip()
            return float(out)
        except (subprocess.SubprocessError, OSError, ValueError):
            return None


    def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
                      out_dir: str, begin_pad: float = BEGIN_PADDING,
                      end_pad: float = END_PADDING) -> List[str]:
        """Write one padded clip per requested rally; returns the clip paths written."""
        if not os.path.exists(video_path):
            raise FileNotFoundError(f"video not found: {video_path}")
        labels = load_rally_labels(labels_csv)
        if not labels:
            raise ValueError(f"no usable rally rows in {labels_csv}")
        os.makedirs(out_dir, exist_ok=True)

        duration = _probe_duration(video_path)
        windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
        if not windows:
            raise ValueError("no rallies matched the request (check --rallies vs the CSV)")

        stem = os.path.splitext(os.path.basename(video_path))[0]
        written: List[str] = []
        for w in windows:
            safe_rally = w.rally.replace(".", "_")
            out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
            cmd = [
                "ffmpeg", "-y",
                "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
                "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
                "-c:a", "aac", "-b:a", "128k",
                out_path,
            ]
            try:
                subprocess.run(cmd, capture_output=True, check=True)
            except subprocess.CalledProcessError as e:
                err = e.stderr.decode(errors="replace").strip()[-300:]
                print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
                continue
            if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
                written.append(out_path)
```

```
154                 print(f"[slicer] rally {w.rally} -> {out_path}
[{w.start:.3f}-{w.end:.3f}s]")
155         return written
156
157
158     def main() -> None:
159         p = argparse.ArgumentParser(description="Slice rally clips from an annotated
video.")
160         p.add_argument("--video", required=True, help="Source video file")
161         p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
162         p.add_argument("--rallies", required=True,
163                        help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
164         p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
165         p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
166         p.add_argument("--end-pad", type=float, default=END_PADDING)
167         args = p.parse_args()
168
169         rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
170         written = slice_rallies(args.video, args.labels, rally_numbers,
171                                 args.out_dir, args.begin_pad, args.end_pad)
172         print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
173
174
175     if __name__ == "__main__":
176         main()
177
```

22. user (2026-06-10T05:34:12.153Z)

```
1       """CLI: regression-test the production tool against golden-set ground truth.
2
3       Obtains ground truth from an annotation provider (a tier), scores the video's
4       stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5       detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7       Examples
8       --------
9       Snapshot the current numbers as the baseline (first run / after a real improvement):
10          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12      Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14      """
15      import os
16      import sys
17      import json
18      import argparse
19      import logging
20
21      from backend.infrastructure.database import Database
22      from backend.config import load_config
23      from backend.annotations import annotation_registry
24      from backend.eval import regression
25
26
27      def _default_db_path() -> str:
28          cfg = load_config()
29          return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32      def build_parser() -> argparse.ArgumentParser:
33          p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34          p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
```

```
predictions to score.")
35          p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37          p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38          p.add_argument("--stage", choices=("candidates", "final"), default="final")
39          p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40          p.add_argument("--iou", type=float, default=0.5)
41          p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42          p.add_argument("--db", default=None)
43          p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44          p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46          p.add_argument("--json", dest="json_out", default=None)
47          return p
48
49
50      def main(argv=None) -> int:
51          logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
52          args = build_parser().parse_args(argv)
53
54          db_path = args.db or _default_db_path()
55          if not os.path.exists(db_path):
56              print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
57              return 2
58          db = Database(db_path)
59
60          result = regression.regress_with_provider(
61              db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
62              stage=args.stage, iou_threshold=args.iou, detector=args.detector,
63              tolerance=args.tolerance, baseline_path=args.baseline,
64          )
65
66          print(regression.format_result(result))
67
68          if args.update_baseline:
69              regression.save_baseline(result.video_id, result.stage, result.precision,
70                                       result.recall, result.f1, path=args.baseline)
71              print(f"Baseline updated in {args.baseline}.")
72
73          if args.json_out:
74              with open(args.json_out, "w") as f:
75                  json.dump(result.as_dict(), f, indent=2)
76              print(f"JSON written to {args.json_out}")
77
78          # Non-zero exit on regression so CI can gate on it.
79          return 1 if result.regressed else 0
80
81
82      if __name__ == "__main__":
83          raise SystemExit(main())
84
```

23. user (2026-06-10T05:34:17.286Z)

```
1       """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3       Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4       API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
```

```
      Examples
      --------
      Create an empty annotation file to fill in:
          python run_eval.py --scaffold --annotations rallies.csv

      Score a video's detected rallies (identify it by file or by stored video_id):
          python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
          python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5

      Show which rallies were missed / spurious, and dump a JSON report:
          python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
      """

      import os
      import sys
      import json
      import argparse
      import logging

      from backend.infrastructure.database import Database
      from backend.config import load_config
      from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
      from backend.eval.annotations import load_annotations, write_scaffold
      from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES


      def _default_db_path() -> str:
          """Resolve the DB path from the typed config (output.output_dir /
output.db_name)."""
          cfg = load_config()
          return os.path.join(cfg.output.output_dir, cfg.output.db_name)


      _EPILOG = """\
      Prerequisite:
        The harness scores predictions read from the SQLite DB ? it does NOT run the
        pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
        so its detections land in output/sports_indexer.db. Then point this tool at the
        same video file (--video) or its stored MD5 (--video-id). If you get
        "database not found" or zero predictions, the video hasn't been processed yet.

      See docs/EVALUATION.md for the annotation format and how to read the output.
      """


      def build_parser() -> argparse.ArgumentParser:
          p = argparse.ArgumentParser(
              description="Evaluate detected rallies vs ground-truth annotations.",
              epilog=_EPILOG,
              formatter_class=argparse.RawDescriptionHelpFormatter,
          )
          p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end
per row).")
          p.add_argument("--video", help="Path to the video file (its MD5 becomes the
video_id).")
          p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of
--video.")
          p.add_argument("--db", default=None, help="SQLite DB path (default: from
config.json).")
          p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
                         help="Which prediction stage(s) to score (default: both).")
          p.add_argument("--detector", default=None,
                         help="Only score candidates from this detector (e.g. yolo_hybrid,
tracknet_hybrid).")
```

```
        p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
        p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep
(0.1..0.9) in the report.")
        p.add_argument("--show-failures", action="store_true",
                       help="List missed (FN) and spurious (FP) rallies with timestamps.")
        p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to
this path.")
        p.add_argument("--scaffold", action="store_true",
                       help="Write an empty annotation CSV to --annotations and exit.")
        return p


    def main(argv=None) -> int:
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        args = build_parser().parse_args(argv)

        # Scaffold mode: just write the template and exit.
        if args.scaffold:
            if not args.annotations:
                print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
                return 2
            write_scaffold(args.annotations)
            print(f"Wrote empty annotation file: {args.annotations}")
            return 0

        if not args.annotations:
            print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
            return 2

        # Resolve the video_id.
        if args.video_id:
            video_id = args.video_id
        elif args.video:
            if not os.path.exists(args.video):
                print(f"ERROR: video not found: {args.video}", file=sys.stderr)
                return 2
            print("Computing video MD5...")
            video_id = get_file_md5(args.video)
            print(f"video_id = {video_id}")
        else:
            print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
            return 2

        db_path = args.db or _default_db_path()
        if not os.path.exists(db_path):
            print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
            return 2

        gt = load_annotations(args.annotations)
        if not gt:
            print(f"WARNING: no ground-truth rallies in {args.annotations}.",
file=sys.stderr)

        stages = list(STAGES) if args.stage == "both" else [args.stage]
        db = Database(db_path)
        report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
                          detector=args.detector, with_pr_curve=args.pr_curve)

        print(format_report_text(report, show_failures=args.show_failures))

        if args.json_out:
            with open(args.json_out, "w") as f:
                json.dump(report_to_dict(report), f, indent=2)
```

```
            print(f"\nJSON report written to {args.json_out}")

        return 0


    if __name__ == "__main__":
        raise SystemExit(main())

```

24. user (2026-06-10T05:34:17.408Z)

```
      """Phase 3 driver: batch process + evaluate a collector export against ground truth.

      Walks the `manifest.json` produced by sports-data-collector's `curate export`,
      processes each video through a segmenter (so detections land in the indexer DB),
      scores each against its ground-truth rally CSV, and prints per-video plus
      corpus-aggregate precision/recall/F1, mean IoU, and boundary error.

      Examples
      --------
      Free CPU baseline on a single video (validate the loop):
          python run_phase3_eval.py --manifest ../sports-data-
collector/data/eval_export/manifest.json \
              --segmenter motion --only shuttleset_1

      Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
          python run_phase3_eval.py --manifest .../manifest.json --segmenter yolo_hybrid \
              --json output/phase3_report.json

      Re-score only (videos already processed), no re-segmentation:
          python run_phase3_eval.py --manifest .../manifest.json --score-only
      """

      import os
      import sys
      import json
      import time
      import argparse
      import logging

      from dotenv import load_dotenv
      load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters

      from backend.infrastructure.database import Database
      from backend.config import load_config
      from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
      # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
      from backend.pipeline.segmenters import segmenter_registry
      from backend.eval.annotations import load_annotations
      from backend.eval.harness import evaluate, format_report_text, report_to_dict
      from backend.eval import batch

      logger = logging.getLogger(__name__)


      def build_parser() -> argparse.ArgumentParser:
          p = argparse.ArgumentParser(
              description="Batch process + evaluate a collector export manifest (Phase 3).",
              formatter_class=argparse.RawDescriptionHelpFormatter,
          )
          p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
          p.add_argument("--videos-root", default=None,
                         help="Root for resolving manifest relative local_paths "
                              "(default: the collector repo root, two levels above the
```

```
manifest).")
53          p.add_argument("--segmenter", default="motion",
54                         help="Segmenter to run (default: motion ? free/CPU). "
55                              "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
56          p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
default="auto",
57                         help="Which prediction stage(s) to score (default: auto by
segmenter).")
58          p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
59          p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
60          p.add_argument("--only", action="append", default=None,
61                         help="Only this video_id (repeatable).")
62          p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
63          p.add_argument("--max-frames", type=int, default=None,
64                         help="Cap segmenter frames (fast validation).")
65          p.add_argument("--score-only", action="store_true",
66                         help="Skip segmentation; only score detections already in the DB.")
67          p.add_argument("--reprocess", action="store_true",
68                         help="Re-run segmentation even if the video already has segments.")
69          p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
70          p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report
here.")
71          p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
72          return p
73
74
75      def main(argv=None) -> int:
76          logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
77          args = build_parser().parse_args(argv)
78
79          manifest_path = os.path.abspath(args.manifest)
80          if not os.path.exists(manifest_path):
81              print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
82              return 2
83          manifest_dir = os.path.dirname(manifest_path)
84          videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
85
86          with open(manifest_path) as f:
87              manifest = json.load(f)
88          entries = manifest.get("eval_sets", [])
89
90          cfg = load_config()  # typed AppConfig (same object the live pipeline feeds
segmenters)
91          db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
92          os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
93          db = Database(db_path)
94
95          stages = batch.default_stages_for(args.segmenter, args.stage)
96
97          # Resolve the segmenter class up front (unless purely scoring).
98          segmenter = None
99          if not args.score_only:
100             seg_cls = segmenter_registry.get(args.segmenter)
101             if not seg_cls:
102                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
103                       f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
104                 return 2
105             segmenter = seg_cls(db, cfg)
106
107         # Filter entries.
```

```
        targets = []
        for e in entries:
            if args.only and e["video_id"] not in args.only:
                continue
            targets.append(e)
        if args.limit:
            targets = targets[: args.limit]

        per_video = []
        skipped = []
        print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")

        for e in targets:
            vid = e["video_id"]
            video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
            csv_path = os.path.join(manifest_dir, e["csv"])

            if not video_path or not os.path.exists(video_path):
                skipped.append((vid, "no video file"))
                print(f"[skip] {vid}: no video file ({e.get('local_path')})")
                continue
            if not os.path.exists(csv_path):
                skipped.append((vid, "no GT csv"))
                print(f"[skip] {vid}: GT csv missing ({csv_path})")
                continue

            video_id = get_file_md5(video_path)

            # Segment unless score-only or already present (and not --reprocess).
            if not args.score_only:
                already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
                if already and not args.reprocess:
                    print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
                else:
                    print(f"[proc] {vid}: segmenting with {args.segmenter}...")
                    db.add_video(video_id, video_path, "processed", [])
                    t0 = time.time()
                    segs = segmenter.process_video(video_id, video_path,
max_frames=args.max_frames)
                    print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")

            gt = load_annotations(csv_path)
            report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
            print(format_report_text(report, show_failures=args.show_failures))
            print()
            d = report_to_dict(report)
            d["collector_video_id"] = vid
            per_video.append(d)

        agg = batch.aggregate(per_video, stages)
        print(batch.format_aggregate(agg))
        if skipped:
            print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))

        if args.json_out:
            with open(args.json_out, "w") as f:
                json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
                           "aggregate": agg, "per_video": per_video,
                           "skipped": skipped}, f, indent=2)
            print(f"\nJSON report written to {args.json_out}")
```

```
167
168         return 0
169
170
171     if __name__ == "__main__":
172         raise SystemExit(main())
173
```

25. user (2026-06-10T05:34:23.771Z)

```
1       import os
2       import sys
3       import json
4       import logging
5       from dotenv import load_dotenv
6
7       load_dotenv()
8
9       # Configure root logger ? all backend modules use logging.getLogger(__name__)
10      logging.basicConfig(
11          level=logging.INFO,
12          format="%(asctime)s [%(name)s] %(levelname)s: %(message)s",
13          datefmt="%H:%M:%S",
14      )
15
16      from backend.infrastructure.database import Database
17      from backend.config import load_config
18      from backend.utils.hashing import compute_video_id
19      from backend.pipeline.validators.base import registry
20      from backend.pipeline.segmenters import segmenter_registry
21      from backend.pipeline.stitcher.compiler import VideoCompiler
22
23      def main():
24          video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
25          output_dir = r"C:\Users\avidu\Videos\Testing"
26          output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
27
28          print(f"Loading config...")
29          cfg = load_config()  # typed AppConfig ? single source of defaults
30
31          print(f"Verifying files exist...")
32          if not os.path.exists(video_path):
33              print(f"Error: Video file not found at {video_path}")
34              return
35
36          os.makedirs(cfg.output.output_dir, exist_ok=True)
37          db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
38          db = Database(db_path)
39
40          # Calculate MD5 of the video to uniquely identify it
41          import time
42          print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
43          t0 = time.time()
44          video_id = compute_video_id(video_path)
45          print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
46
47          segments = []
48          cached_video = db.get_video(video_id)
49          cache_hit = False
50
51          if cached_video and cached_video.get("status") == "processed":
52              segments = db.query_play_segments(video_id, {})
53              if len(segments) > 0:
54                  print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
```

```
with {len(segments)} parsed play segments.", flush=True)
                choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
                if choice == '1':
                    print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
                    cache_hit = True
                else:
                    print("[CACHE] Ignoring cache...", flush=True)
            else:
                print(f"[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)

        if not cache_hit:
            print("[CACHE] Initializing clean indexing...", flush=True)
            db.clear_video_data(video_id)

            print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
            db.add_video(video_id, video_path, "processed", [])

            print("Running segmenter & indexing...", flush=True)
            seg_name = cfg.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return
            print(f"Using segmenter: {seg_name}", flush=True)
            segmenter = segmenter_cls(db, cfg)
            segments = segmenter.process_video(video_id, video_path)
            print(f"Successfully indexed {len(segments)} play segments.", flush=True)

        if not segments:
            print("No play segments detected. Cannot compile highlights.")
            return


        end_padding = cfg.stitching.end_padding
        begin_padding = cfg.stitching.begin_padding
        min_shot_count = cfg.stitching.min_shot_count

        if cache_hit:
            print("\n--- Stitching Customizations ---")
            try:
                bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
                if bpad_input:
                    begin_padding = float(bpad_input)

                pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
                if pad_input:
                    end_padding = float(pad_input)

                shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
                if shot_input:
                    min_shot_count = int(shot_input)
            except ValueError:
                print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")

        print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)

```

```
        # Filter segments where shot_count >= min_shot_count
        time_pairs = []
        skipped_count = 0
        for s in segments:
            meta = s.get("metadata", {})
            if isinstance(meta, str):
                try:
                    meta = json.loads(meta)
                except Exception:
                    meta = {}
            shot_count = meta.get("shot_count", 0)
            if shot_count >= min_shot_count:
                time_pairs.append((s["start_time"], s["end_time"]))
            else:
                skipped_count += 1

        if skipped_count > 0:
            print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.", flush=True)
        print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
flush=True)

        if not time_pairs:
            print("No play segments passed the filtering criteria. Cannot compile
highlights.")
            return

        compiler = VideoCompiler(output_dir, end_padding=end_padding,
begin_padding=begin_padding)
        try:
            out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
            print(f"Stitched highlight video saved to: {out_path}", flush=True)
        except Exception as e:
            print(f"Error during stitching: {e}", flush=True)

    if __name__ == "__main__":
        main()
```

26. user (2026-06-10T05:34:24.073Z)

```
      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.

      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
      the opponent for service after losing a point, that single trajectory looks like a
      rally. The discriminator is structural, not duration-based: a real rally has the
      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
      single service feed crosses ~once.

      This module is camera-agnostic and needs NO new model ? it works on the trajectory
      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
      axis with the larger shuttle spread) and the net position (median shuttle coord on
      that axis ? the shuttle clusters at the two ends and crosses the net region often,
      so the median sits near the net), then counts sign changes of (coord - net) across
      the visible points inside a candidate window, with a deadband to ignore jitter.

      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
      windows from trajectory_to_action_windows, or to fold into it later.
      """
      from __future__ import annotations

      from dataclasses import dataclass
      from typing import List, Sequence, Tuple

```

```
    Interval = Tuple[float, float]


    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]

```

```
90
91      def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93          """Annotate each (start,end) window with its shuttle net-crossing count and a
94          keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95          axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96          axis, net, span = estimate_play_axis(points)
97          deadband = deadband_frac * span
98          out: List[GatedWindow] = []
99          for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                    kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113
```

27. user (2026-06-10T05:34:30.704Z)

```
1       """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4       inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6           1. Translate the Windows video path to its WSL-visible form (and optionally
7              stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8           2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9           3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10          4. Convert that trajectory into high-motion "action windows" ? the same
11             candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12             pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
```

```
        weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
        queue_length: int = 5
        stage_in_wsl: bool = True                       # copy video into WSL fs first
(perf)
        wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
        timeout_sec: int = 0                            # 0 = no timeout

        @classmethod
        def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
            tn = (idx_cfg or {}).get("tracknet", {}) or {}
            return cls(
                distro=tn.get("wsl_distro", "Ubuntu"),
                repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
                conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
                conda_env=tn.get("conda_env", "TrackNetV4"),
                weights_path=tn.get("weights_path", ""),
                queue_length=int(tn.get("queue_length", 5)),
                stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
                wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
                timeout_sec=int(tn.get("timeout_sec", 0)),
            )


    def to_wsl_mnt_path(win_path: str) -> str:
        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).

        Already-POSIX paths (starting with / or ~) are returned unchanged so the
        same helper is safe to call on either kind of path.
        """
        p = str(win_path)
        if p.startswith("/") or p.startswith("~"):
            return p
        p = p.replace("\\", "/")
        if len(p) >= 2 and p[1] == ":":
            drive = p[0].lower()
            return f"/mnt/{drive}{p[2:]}"
        return p


    @dataclass
    class TrajectoryPoint:
        frame: int
        visible: bool
        x: float
        y: float


    class TrackNetRunner(DetectorRunner):  # DetectorRunner ABC (finish item 6 / new item 1)
        """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.

        Implements the common DetectorRunner contract so a future Linux-native or
        alternative trajectory runner can be substituted without touching the hybrids.
        """

        def __init__(self, cfg: TrackNetConfig):
            self.cfg = cfg

        # ------------------------------------------------------------------ WSL exec

        def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
            """Run a bash command string inside the configured WSL distro."""
            full = f"source {self.cfg.conda_sh} && {inner_cmd}"
            argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
```

```
99              logger.debug("WSL exec: %s", full)
100             return subprocess.run(
101                 argv, capture_output=True, text=True,
102                 timeout=(self.cfg.timeout_sec or None),
103             )
104
105         def healthcheck(self) -> Tuple[bool, str]:
106             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
107
108             Returns (ok, message). Cheap to call before a long run.
109             """
110             cmd = (
111                 f"conda activate {self.cfg.conda_env} && "
112                 f"python -c \"import tensorflow as tf; "
113                 f"print('TF', tf.__version__, 'GPUs',
len(tf.config.list_physical_devices('GPU')))\""
114             )
115             try:
116                 res = self._wsl_bash(cmd)
117             except FileNotFoundError:
118                 return False, "`wsl` not found ? is this running on Windows with WSL2
installed?"
119             except subprocess.TimeoutExpired:
120                 return False, "WSL healthcheck timed out."
```

28. user (2026-06-10T05:34:32.099Z)

```
213:    def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
232:    def trajectory_to_action_windows(
```

29. user (2026-06-10T05:34:37.092Z)

```
205             if res.returncode != 0 or "OK" not in (res.stdout or ""):
206                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
207                 return None
208             return dest
209
210         # ------------------------------------------------------------- CSV -> windows
211
212         @staticmethod
213         def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
214             """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
215             points: List[TrajectoryPoint] = []
216             with open(csv_win_path, newline="") as f:
217                 reader = csv.DictReader(f)
218                 for row in reader:
219                     try:
220                         points.append(TrajectoryPoint(
221                             frame=int(row["Frame"]),
222                             visible=int(row["Visibility"]) == 1,
223                             x=float(row["X"]),
224                             y=float(row["Y"]),
225                         ))
226                     except (KeyError, ValueError):
227                         continue
228             points.sort(key=lambda p: p.frame)
229             return points
230
231         @staticmethod
232         def trajectory_to_action_windows(
233             points: List[TrajectoryPoint],
234             fps: float,
235             frame_width: float,
236             chunk_start: float = 0.0,
```

```
            velocity_thresh: float = 0.02,
            merge_gap: float = 2.0,
            min_window_duration: float = 2.0,
            chunk_index: Optional[int] = None,
        ) -> List[Dict[str, Any]]:
            """Convert a shuttle trajectory into candidate rally windows.

            A frame is "active" when the shuttle is visible AND its inter-frame
            displacement (normalised by frame width, per second) exceeds
            ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
            ``merge_gap`` seconds of each other are grouped into one window; short
            windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
            shape feeding the AI-handoff phase is identical.
            """
            if fps <= 0:
                fps = 30.0
            if frame_width <= 0:
                frame_width = 1280.0

            # Compute per-frame normalised velocity from the last *visible* point.
            active = []  # (frame_idx, velocity)
            prev: Optional[TrajectoryPoint] = None
            for p in points:
                if not p.visible:
                    prev = None  # break the trajectory; absent shuttle = not in flight
                    continue
                if prev is not None:
                    dframes = p.frame - prev.frame
                    if dframes > 0:
                        dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
                        velocity = (dist / frame_width) * (fps / dframes)
                        if velocity > velocity_thresh:
                            active.append((p.frame, velocity))
                prev = p

            if not active:
                return []

            max_gap_frames = int(merge_gap * fps)

            def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
                vels = [v for _, v in group]
                return {
                    "start": group[0][0] / fps + chunk_start,
                    "end": group[-1][0] / fps + chunk_start,
                    "active_frame_count": len(group),
                    "peak_velocity": max(vels),
                    "mean_velocity": sum(vels) / len(vels),
                    "track_id_count": 1,  # single shuttle; kept for window-shape parity
                    "chunk_index": chunk_index,
                }

            windows = []
            group = [active[0]]
            for af in active[1:]:
                if af[0] - group[-1][0] <= max_gap_frames:
                    group.append(af)
                else:
                    windows.append(_finalize(group))
                    group = [af]
            windows.append(_finalize(group))

            return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

```

30. assistant (2026-06-10T05:34:42.282Z)

Now the owned-model plan Phase-0 and the multivideo LOVO docs to ground the methodology assessment.

31. user (2026-06-10T05:34:44.328Z)

46:- **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss strict temporal boundaries**
58:- **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits structured timestamped
72:- **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned across sports (sport = an input
99:## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
105:labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL ActivityNet-JSON · VLM clip?QA),
120: per sport.** Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
122:### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
131:## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
132:Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it becomes the rally/highlight
150:| 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4 harness** | awaiting owner |
157:## Open risks / unknowns (none block Phase-0 research)

32. user (2026-06-10T05:34:45.736Z)

README.md
__init__.py
__pycache__
test_annotation_providers.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_config.py
test_database.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_training.py
test_export_wasb_segments.py
test_gemini.py
test_gemini_refine.py
test_geometry.py
test_hashing.py
test_hit_detector.py
test_provider_gemini.py
test_providers.py
test_rally_gate.py
test_rally_seq_proto.py
test_rally_slicer.py
test_reporting.py
test_reporting_harvest.py
test_reporting_spec_faq.py
test_sport_profiles.py
test_tracknet_hybrid.py
test_tracknet_runner.py
test_validators_base.py

```
test_video.py
test_wasb_cache.py
test_yolo_hybrid.py
```

33. user (2026-06-10T05:34:50.337Z)

```
95        separate service. **None exists today.**
96
97      ---
98
99      ## Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)
100
101     ### M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) ·
*greenlight item*
102     Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index
schema (the conversational seam)
103     ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end,
ending_reason/fault_tags,
104     shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY
MATCH), trajectory_ref,
105     labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL
ActivityNet-JSON · VLM clip?QA),
106     one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector`
(system-of-record; extends PR #13).
107     **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match
enforced; **no Gemini-sourced rows in any training view**.
108
109     ### M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
110     6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-
match + LOVO recorded; the
111     "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club
match ? margin. **Bottleneck =
112     data + calibration, not method.** Keep growing per new sport.
113
114     ### M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) ·
*greenlight item*
115     - **(i) PURGE the Gemini-derived TRAINING labels** in `training.label_candidates` (over
Gemini CSVs) ? a **live
116       no-paid-LLM-training violation** that bakes in the moment a training run precedes it.
Retarget to human + open-teacher
117       (own-model/WASB) labels; Gemini = eval/reference/fallback only.
118     - **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint
provenance is a **commercial-ship
119       blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a
clean detector swap; **multiplies
120       per sport.** Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).
121
122     ### M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
123     Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over
**ActionFormer (MIT, 1:1
124     rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not*
the Commons-Clause repo),
125     reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item):
a FLOOR of "beat the honest n=6
126     heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU**
target for "very high P/R" (e.g.
127     F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory.
Trains on the GPU/WSL box.
128
129     ---
130
131     ## Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
132     Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it
becomes the rally/highlight
133     extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no
```

```
overfitting to declare victory.
134
135     ## Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus +
explicit go)
136     - **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a
promising cue here** ? our E1 probe
137       (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea
also failed this session; keep it
138       parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet
first).
139     - **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a
temporal-IoU-vs-golden reward,
140       our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on
Modal (no-access) / RunPod-Secure.
141     - **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-
hosted VLM vs Gemini) **+ a real
142       VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples
use 6?8 GPUs ? **budget is the
143       largest open risk and is currently unquantified**). Don't stand up an owned serving
stack until these clear.
144
145     ---
146
147     ## Decisions / status
148     | # | item | status |
149     |---|---|---|
150     | 1 | Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4
harness** | awaiting owner |
151     | 2 | WASB C3 ? first-class action item (own-weights / clean swap) | **owner: yes** ? |
152     | 3 | Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights) | **owner: yes** ?
|
153     | 4 | Collector = corpus + event-store system-of-record | **owner: confirm?** |
154     | 5 | Rename repo off "badminton" | **owner action** ? |
155     | 6 | Conversational-QA: build seam now, punt model | **rec accepted?** |
156
157     ## Open risks / unknowns (none block Phase-0 research)
158     Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R"
target undefined · per-sport clean
159     trajectory detectors not yet identified · base-model pretraining provenance unauditable
(risk-acceptance) · real VLM
160     VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-
train (minors-excluded) for cloud
161     training · on-device 4B latency on 8 GB unmeasured.
162
163     ---
164     *Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing
discipline holds: the committed
165     next platform slice remains the async job model (single-tenant); only the M0.1?M0.4
foundation is safe to start.*
166
```

34. user (2026-06-10T05:37:13.670Z)

```
2026-06-human-gt-tuning
2026-06-initial
2026-06-multivideo-lovo
README.md
README.md
```

35. user (2026-06-10T05:37:14.411Z)

```
1       """Rally-detection evaluation harness.
2
3       Scores the pipeline's detected rallies against hand-annotated `[start, end)`
4       ground-truth rally timestamps, using temporal interval matching. See
```

```
5       docs/EVALUATION.md for the annotation format and CLI usage.
6
7       - `metrics`     ? pure interval-matching math (IoU, matching, P/R/F1, boundary error).
8       - `annotations` ? load ground-truth rally CSVs.
9       - `harness`     ? read predictions from the DB and score them.
10      """
11
```

36. user (2026-06-10T05:37:15.706Z)

```
1:import os
2:import time
3:import json
4:import logging
5:from typing import Tuple
6:from google import genai
7:from google.genai import types
8:from backend.providers.base import AIVideoProvider
17:    def __init__(self, api_key: str, model_name: str):
```

37. user (2026-06-10T05:37:23.640Z)

```
1       from typing import Dict, Type
2       from backend.providers.base import AIVideoProvider
3       from backend.providers.gemini import GeminiProvider
4
5       class ProviderRegistry:
6           def __init__(self):
7               self._registry: Dict[str, Type[AIVideoProvider]] = {}
8               # Register Gemini provider
9               self.register("gemini", GeminiProvider)
10
11          def register(self, name: str, provider_cls: Type[AIVideoProvider]):
12              self._registry[name] = provider_cls
13
14          def get(self, name: str, api_key: str, model_name: str) -> AIVideoProvider:
15              if name not in self._registry:
16                  raise KeyError(f"AI Provider '{name}' is not registered in the
ProviderRegistry. Available: {list(self._registry.keys())}")
17              return self._registry[name](api_key=api_key, model_name=model_name)
18
19          def list_available(self) -> Dict[str, str]:
20              return {name: cls.__doc__ or f"{name.capitalize()} provider" for name, cls in
self._registry.items()}
21
22      provider_registry = ProviderRegistry()
23
```

38. user (2026-06-10T05:37:24.462Z)

```
1       # Quality Iteration: 2026-06 Multi-video LOVO + learned-segmenter prototype
2
3       **Date:** 2026-06-08
4       **STATUS: COMPLETED** (second concrete iteration; follows `../2026-06-human-gt-tuning/`)
5
6       > The session that turned "tune one match" into "is the *method* right across matches."
Built a 2-match
7       > human-labeled corpus, ran the first honest cross-video LOVO, uncovered that the
unsupervised windowing
8       > heuristic is **footage-fragile even when WASB recall is excellent**, and **proved with
a prototype that the
9       > rally signal is learnable** where the heuristic fails. Motivates + de-risks the owned
model
10      > (`../../research/OWNED_MODEL_TRAINING_STUDY.md`).
```

11
12   ## Corpus built this iteration
13 | video | role | rallies | res / fps | trajectory |
14 |---|---|---|---|---|
15 | Adarsh-and-Avi (full) | train | **96** | singles · 1080p / 59.94 |
`~/clips/adarsh_avi_full_traj.csv` (84,480 fr) |
16 | Kushagra-Yash-Avi | train | 32 | singles · 4K?1920 / 59.94 |
`~/clips/kushagra_traj.csv` (32,232 fr) |
17 | **LargeTestVideo** | **train** | **49** | **doubles · 5.3K?1920 / 29.97** |
`~/clips/largetestvideo_traj.csv` (23,040 fr) |
18 | **gBaaddy (20 May)** | **train** | **47** | **1080p / 59.94 · multi-court** |
`~/clips/gbaaddy_traj.csv` (52,232 fr) |
19 | **TestLargeVideo** | **train** | **6** | **5.3K?1920 / 29.97 · short** |
`~/clips/testlarge_traj.csv` (3,019 fr) |
20
21   *(Grew to **n=5 distinct matches** over the session: 2 singles + 1 doubles + 2 more,
across 3 cameras/frame-rates,
22   incl. 2 multi-court ? see the n=3 and n=5 updates below.)*
23
24   All WASB trajectories generated on one 8 GB GPU, **serialized via file-based wait-
loops** (`run_wasb_*.sh`;
25   GPU never double-booked, race-free via a `kushagra_traj.csv`-exists gate). Manifest:
`output/human_lovo_manifest.json`.
26
27   ## Result 1 ? full-match Adarsh (96 rallies, IoU?0.5)
28   baseline F1 **0.632**, tuned+gate2 **0.727** (P0.80/R0.67), sweep ceiling **0.750**
(vel0.02/merge1.0/min_dur2.0/gate off).
29   vs the opening-10-min slice (40 rallies, prior iteration): 0.667 / 0.742. **Best-config
region is identical across the
30   slice and the full match** (merge_gap 1.0, min_dur 2.0; velocity irrelevant) ? tuning is
stable. Per-rally:
31   `output/adarsh_avi_full_vs_human.csv`.
32
33   ## Result 2 ? first cross-video LOVO (`calibrate_local`, the number to trust)
34   ```
35   per-video best-fit: adarsh 0.777 | kushagra 0.163 (ceiling ? NO config beats it)
36   leave-one-video-out: baseline mean 0.316 ? calibrated 0.443 ± 0.279 (overfit gap
+0.027, tiny)
37   held-out adarsh   0.722   (config tuned on Kushagra ? Adarsh's own optimum ? config
TRANSFERS)
38   held-out kushagra 0.163   (config tuned on Adarsh = Kushagra's own ceiling)
39   ```
40   **The asymmetry is the finding.** Tuning generalizes fine *where the heuristic can solve
the video*
41   (Kushagra?Adarsh = 0.722). But **Kushagra is unsegmentable by *any* velocity-windowing
config (0.163)** ? and
42   this is NOT a WASB-recall failure: WASB tracks the shuttle **better** on Kushagra than
Adarsh (**97% of GT
43   rallies contain ?2 net-crossings**, median 5, 310 visible pts/rally, vs Adarsh 76%). The
likely mechanism:
44   Kushagra's shuttle is visible 67.5% of the time (vs Adarsh 40%) ? tracked *through*
inter-rally activity ? the
45   "shuttle-is-moving" heuristic can't place clean boundaries (mean boundary error ~5 s).
**The ±0.279 spread is
46   method variance, not config.** This also **corrects the earlier "~0.73 baseline"** ?
that was fit-on-self; the
47   honest cross-video heuristic bar is **~0.44.**
48
49   ## Result 3 ? learned per-frame prototype (the proof) ?
`backend/eval/rally_seq_proto.py`
50   A dependency-light prototype (numpy + scipy + the repo's pure-Python
`LogisticRegression`): 6 hand-built per-frame
51   features over the WASB trajectory ? visibility rate, mean/max shuttle speed, **net-
crossing rate**, x/y spread ?
52   labeled rally(1)/non-rally(0), **honest leave-one-video-out** (train on one match, test

the other; split by match).

held-out	per-frame AUC	learned F1 (real LOVO)	learned F1 (oracle threshold)	heuristic ceiling
Adarsh	0.878	0.704	0.721	0.73
Kushagra	**0.841**	0.049	**0.578**	**0.163**

The signal is learnable. Held-out per-frame AUC is **0.84?0.88** ? including on Kushagra (trained only on Adarsh), the match the heuristic *cannot* segment. With a calibrated threshold the learned model hits **Kushagra F1 0.578 ? 3.5× the heuristic's 0.163 ceiling.** The naive cross-video threshold gives 0.049, so the remaining gap is **threshold/operating-point calibration at n=2, not representation** (the probability scale shifts across footage; fixed by more videos / per-video calibration / a scale-invariant sequence model).

> This directly answers the owned-model study's "dominant empirical unknown" ? *can a low-dim WASB trajectory feed a learned temporal segmenter?* **Yes (AUC 0.84).** The prototype is the seed of the study's **Gen-0 (ASFormer-over-WASB-features)**. ? It is a *directional* proof, not production: n=2, linear model, hand features, threshold-transfer unsolved. The real ship gate is a segment-level A/B on the golden set at n?5 (needs the GPU/WSL machine ? not runnable in this dev container).

Update ? n=3: doubles match added ? **the crossover** (the final number)

Added **LargeTestVideo** (doubles, 5.3K?1920, **29.97 fps**, 49 rallies, fully tagged) ? a different play type *and* camera *and* frame rate. Single-video: heuristic best F1 **0.750** (tuned, R 0.918), best-fit 0.827 ? so **doubles is heuristic-friendly; Kushagra remains the lone heuristic failure.** Per-rally: `output/largetestvideo_vs_human.csv`. Manifest now n=3 (`output/human_lovo_manifest.json`).

Honest cross-match LOVO, heuristic vs learned, n=2 ? n=3:

held-out match	heuristic LOVO	learned LOVO (AUC)	learned LOVO F1
Adarsh (singles)	0.725	0.878	0.701
Kushagra (singles, the hard one)	**0.151**	0.850	**0.417**
LargeTest (doubles)	0.755	**0.914**	0.729
mean held-out F1	**0.544 ± 0.278**	?	**0.615**
(same metric at n=2)	*0.443*	?	*0.377*

Findings (the data-backed case, honestly):

1. **The learned model OVERTAKES the heuristic at n=3** (mean **0.615 vs 0.544**); at n=2 it *trailed* (0.377 vs 0.443). The crossover the plan predicted ? the learned path **scales with data**, the heuristic does not.
2. **Both means rose with n, for opposite reasons.** The heuristic rose only because the new match (doubles) happens to suit it; it **still cannot rescue Kushagra at any n** (held-out 0.151, capped). The learned model rose because **more data closed its calibration gap** ? Kushagra's held-out F1 went **0.049 ? 0.417** just from one added match (oracle ceiling 0.593, AUC 0.850 ? it will keep climbing with n).
3. **The decisive edge is on the *hard* footage** ? where the heuristic fails

(Kushagra), the learned model is
97   **2.8× better and rising**; on the easy matches (Adarsh, doubles) they're comparable
(~0.72?0.75). The owned
98   model's value is exactly the generalization the heuristic lacks.
99   4. **Generalizes across play type** ? the **doubles** match, held out and trained only
on the **two singles**
100   matches, scores AUC **0.914** / F1 0.729. Different player count, camera, and frame
rate, and the per-frame
101   rally signal still transfers. (Notably *despite* the prototype's speed features not
yet being fps-normalized
102   ? 30 vs 60 fps ? which is a known refinement that should only help.)
103
104   ### Final quality metrics (n=3, honest leave-one-video-out)
105   - **Heuristic (current pipeline):** mean held-out F1 **0.544 ± 0.278** ? the ±0.278 is
the footage-fragility
106   (Kushagra ~0.15 vs others ~0.75); **hard-capped, does not improve on the videos it
can't represent.**
107   - **Learned prototype (owned-model seed):** mean held-out F1 **0.615**, per-frame **AUC
0.85?0.91**, **+0.24
108   for one added match and still climbing.** This is a *directional* number (n=3, linear
model, hand features,
109   fps-norm + per-video calibration still on the table) ? the trend, not the absolute, is
the signal.
110
111   **Conclusion (reassures the plan ? with data, not optimism):** the owned/learned path
has **already overtaken
112   the heuristic at n=3 and improves with every match**, while the heuristic is **ceilinged
by footage it cannot
113   represent**. The bottleneck is confirmed to be **data scale + calibration (n?5), not
method.** Reusing every
114   labeled video for training is the right strategy ? each one lifts the learned model and
none can fix the
115   heuristic's hard cases. ? Proceed with `../../research/OWNED_MODEL_TRAINING_STUDY.md`
and
116   `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (Gen-0 first), gated on n?5 + owner sign-off.
117
118   ## Update ? n=5 + the MULTI-COURT root cause (the defining finding)
119
120   Added **gBaaddy** (47 rallies, 1080p/60) + **TestLargeVideo** (6 rallies, 5.3K/30 ? *too
short to be a reliable
121   fold*). Corpus now n=5. Single-video heuristic: gBaaddy **0.277**, TestLarge 0.316 ?
both low. gBaaddy is 1080p/60
122   *like Adarsh* yet fails, so it's **not** a resolution/fps issue.
123
124   ### Root cause ? confirmed with data (owner's insight): background games
125   WASB tracks **every** shuttle in frame, including **other courts**. Videos with
background games have high shuttle
126   visibility *during the foreground game's dead time* ? the velocity-windowing heuristic
can't place clean boundaries.
127   The **"contrast"** metric (in-rally ÷ dead-time shuttle visibility) cleanly predicts
heuristic success:
128
129 | match | heuristic best F1 | in-rally vis | **dead-time vis** | contrast | court |
130 |---|---|---|---|---|---|
131 | Adarsh (singles) | 0.73 | 69% | 18% | **3.9×** | single |
132 | LargeTest (doubles) | 0.75 | 78% | 22% | **3.6×** | single |
133 | Mahadevpura (singles, +n=6) | 0.67 | 74% | 33% | **2.2×** | borderline (some
background) |
134 | gBaaddy | 0.28 | 78% | **41%** | 1.9× | **multi (background games)** |
135 | Kushagra | 0.16 | 82% | **57%** | 1.4× | **multi (background games)** |
136
137   The contrast metric predicts heuristic F1 **monotonically across 5 matches** (3.9×?0.73
? 1.4×?0.16) ? Mahadevpura's
138   intermediate 2.2× ? 0.67 lands exactly on the curve. ?3.6× ? heuristic works; ?1.9× ?
fails; ~2× ? borderline. **Academies are multi-court by definition**, so Kushagra/gBaaddy are

```
the
139     **most market-representative** videos ? "focus on the prominent game" is a core product
requirement, not an edge case.
140     **Lever = foreground-court isolation** (spatial gate to the prominent court before
windowing). The dead-time
141     detections are laterally offset (side courts), but isolation needs **real court-region
detection** ? a naive centroid
142     is muddied because the foreground shuttle also rests low between points. Near-term, no
new model; also a *design
143     requirement* for the e2e owned model (which would learn foreground attention natively).
144
145     ### Learned prototype LOVO (n=5)
146     | held-out | per-frame AUC | learned F1 (LOVO) | oracle F1 | heuristic best |
147     |---|---|---|---|---|
148     | Adarsh (singles) | 0.879 | 0.709 | 0.722 | 0.73 |
149     | Kushagra (multi) | 0.853 | **0.561** | 0.633 | 0.16 |
150     | LargeTest (doubles) | 0.915 | 0.713 | 0.756 | 0.75 |
151     | gBaaddy (multi) | 0.833 | **0.574** | 0.574 | 0.28 |
152     | TestLarge (6 rallies) | 0.726 | 0.286 | 0.471 | 0.32 |
153     | **mean** | ? | **0.568** | ? | ? |
154
155     - **Recovers BOTH multi-court videos** (Kushagra 0.561, gBaaddy 0.574 vs heuristic
0.16/0.28) ? the learned model's
156       spatial-spread features (`x_std`/`y_std`) help discount background courts.
157     - **Kushagra keeps climbing with n:** 0.049 ? 0.417 ? **0.561**.
158     - ? **Honest mean nuance:** the learned mean *dipped* 0.615 (n=3) ? 0.568 (n=5) ?
**entirely the 6-rally TestLargeVideo
159       fold** (F1 0.286, far too short to read). **Excluding it: 0.639** ? above n=3. The
mean is a blunt tool across folds
160       ranging 6?96 rallies; the **per-video learned-vs-heuristic gap on the hard videos is
the real signal**, and it's decisive.
161
162     ### Final quality metrics & conclusion (n=6)
163     **Honest cross-video LOVO mean, heuristic vs learned, by corpus size:**
164
165     | | n=2 | n=3 | n=5 | **n=6** |
166     |---|---|---|---|---|
167     | heuristic (`calibrate_local`) | 0.443 | 0.544 | 0.463 | **0.480 ± 0.242** |
168     | learned prototype | 0.377 | 0.615 | 0.568 | **0.583** (~0.64 excl. 6-rally fold) |
169
170     - **The learned model has led since n=3 and holds ~+0.10** (n=6: learned 0.583 vs
heuristic 0.480). It stays robust as
171       the corpus diversifies; the heuristic stays capped by the multi-court videos it can't
represent (per-fold n=6: Adarsh
172       0.751, doubles 0.737, Mahadevpura 0.657, but Kushagra **0.157** / gBaaddy **0.309**).
173     - **n=6 added MahadevpuraSingles** (borderline multi-court, contrast 2.2×): held out and
trained on the *other 5*, the
174       learned model hit **AUC 0.902 / F1 0.712** (vs heuristic 0.657) ? clean generalization
to an unseen match.
175     - **The decoupling is the headline:** learned held-out **AUC is 0.83?0.92 on every match
*regardless of contrast***,
176       while the heuristic swings 0.16?0.75 *with* contrast. The learned model has
**decoupled from the multi-court problem**
177       the heuristic can't escape.
178     - **The #1 near-term lever is the multi-court confound** (foreground-court isolation) ?
both the heuristic's biggest
179       failure and the product's core requirement. Bottleneck = **data scale + calibration +
multi-court isolation, not method.**
180
181     ## Heuristic discovery for multi-court (what we tried, what's promising)
182
183     Motivated by the contrast finding, two label-free trajectory pre-filters were prototyped
+ measured at the tuned
184     operating point across all 5 videos (scripts not committed ? discovery):
185
```

candidate	multi-court ? (Kushagra/gBaaddy)	clean ? (Adarsh/doubles)	verdict
Foreground spatial gate (keep dominant 2D-density court region)	+0.03 / +0.03	**?0.13 / ?0.12**	? net ?0.04 ? helps multi-court a little but removes legit wide foreground points on clean videos
Track-continuity filter (drop implausible position jumps)	0.00	0.00	? no-op ? the confusion isn't transient jumps; WASB tracks a court coherently, switching at gaps

Conclusion: simple single-cue trajectory heuristics **cannot** isolate the foreground court ? courts aren't separable by raw density, and the learned model already handles multi-court better (AUC 0.83) by combining cues.

Audio for foreground discrimination ? TESTED (2026-06-08), WEAK ?. Hypothesis: the GoPro mic sits at the foreground court, so foreground hits are loud/near and background games quiet/distant ? audio energy gates *"is the prominent game active?"* ? the signal the visual trajectory can't give. **Probed on the 2 multi-court videos + a clean control** (extract audio ? onset-flux + RMS energy, AUC of in-rally vs dead-time): onset/RMS AUC only **0.53?0.65**; audio contrast 1.1?1.9× (below the visual contrast). High-frequency bands (8?22 kHz @ 48 kHz) raise the *contrast* (Kushagra 1.1?1.7×, gBaaddy 1.3?1.9× ? confirming HF-attenuates-with-distance physics) but **AUC stays ~0.6**. **Why it fails:** "dead time" isn't quiet ? the FOREGROUND court is loud during dead-time too (footwork, talk, shuttle pickup) + gym reverb + background games, so audio loudness ? *court occupancy*, not *rally*. Same wall as E1 (~2.2× ceiling).

Verdict: audio alone is NOT the multi-court gate. (A learned timbre/rhythm model or audio *fused into* the learned segmenter might add marginal value, but the visual learned model already hits AUC 0.85 on these videos.)

Court-region detection ? SMOKE-TESTED (2026-06-08), LOW HEADROOM ?. Feasibility probe: of the DEAD-TIME shuttle detections, what fraction fall OUTSIDE the foreground rally region (= separable background courts a court gate could remove)? **Only 12?14% on the multi-court videos** (Kushagra 12%, gBaaddy 14%) ? **86?88% of the dead-time confusion is INSIDE the foreground court region.** A *perfect* court gate lifts contrast only to ~1.6?2.2× (Kushagra 1.4?1.6, gBaaddy 1.9?2.2) ? still below the ?3.6× "works" threshold. So the confusion is **not** spatially-separable background courts; it's the **foreground court's own between-point activity** (footwork, serves, shuttle pickup) and/or background courts that overlap the foreground image area. ? **Court-region detection (heuristic OR a dedicated model) would NOT meaningfully help.** (Doesn't need an own model ? but isn't worth building either.)

The verdict on heuristics: it's a TEMPORAL problem, only the learned model solves it

Three heuristic ideas were tested this session and all came up short ? **foreground spatial gate** (net ?0.04), **audio energy** (AUC ~0.6), **court-region separability** (12?14% out-of-region, low headroom). They fail for the SAME reason: the multi-court dead-time confusion is **rally-vs-non-rally activity in the *same* place and frequency band**, not a spatial or loudness difference. Distinguishing it needs the learned model's **temporal/structural** features (sustained net-crossing rhythm, activity duration, spatial-spread dynamics) ? which is exactly why the prototype already hits **AUC 0.83** on Kushagra/gBaaddy where every hand-crafted heuristic fails. **There is no cheap heuristic shortcut; the learned segmenter is the path.** (A multi-feature heuristic combining cross-rate + speed + spread would just be re-deriving the

```
222     prototype by hand ? so prefer the prototype.)
223
224     ## Learnings (carry forward)
225     - **The bottleneck is the method + data, not WASB recall or labels.** WASB delivers
strong shuttle signal on both
226       matches; the unsupervised heuristic extracts it on one and not the other. A learned
model does (AUC 0.84).
227     - **Tuning is real but footage-bounded** ? `merge_gap 1.0 / min_dur 2.0` transfers; the
gate trades recall?precision;
228       velocity is ~irrelevant. But no config rescues a video the heuristic class can't
represent.
229     - **Calibration, not representation, is the n=2 wall** ? learned probabilities separate
(AUC 0.84) but the operating
230       point doesn't transfer across footage with one training video. **n?5 distinct matches
is the highest-ROI unblock.**
231     - **Group by match, not file** for any cross-val (Adarsh-main + Adarsh-last-game = one
unit; `calibrate_local` has no
232       grouping, so keep one entry per distinct match).
233     - **Reuse the labels** ? Kushagra's labels + trajectory are valuable *training data for
the learned model* even though
234       the current pipeline scores 0.16 on it. Validated empirically.
235
236     ## Reproduce
237     ```bash
238     # trajectories: bash ~/clips/run_wasb_{avi_full,kushagra,gbaaddy}.sh   (serialized on
one GPU)
239     python -m backend.eval.eval_partial_labels --trajectory
"%TEMP%/adarsh_avi_full_traj.csv" \
240       --labels "?/Adarsh and Avi.rallies.csv" --fps 59.94 --frame-width 1920            #
full-match + --sweep
241     python -m backend.eval.calibrate_local  --manifest output/human_lovo_manifest.json
--metric f1   # heuristic LOVO
242     python -m backend.eval.rally_seq_proto  --manifest output/human_lovo_manifest.json
# learned prototype
243     ```
244
245     ## What's next
246     - **? Foreground-court isolation experiment** (the multi-court fix) ? spatially gate the
trajectory to the prominent
247       court before windowing; needs real court-region detection (densest player/shuttle
cluster, court-line detection, or
248       largest-court region), NOT a naive centroid. Would likely fix the heuristic on
Kushagra/gBaaddy AND clean the learned
249       features. Near-term, no new model. Highest-value lever given the academy/multi-court
target env.
250     - **Keep growing the corpus** (more varied matches tomorrow) ? re-run LOVO + prototype;
per-video threshold calibration
251       as n grows. Add fps-normalization to the prototype's speed features.
252     - Build the corpus format + the ASFormer Gen-0 A/B (owner sign-off; per
`../../research/OWNED_MODEL_TRAINING_STUDY.md`
253       + `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`, after scaffolding).
254
255     ## Related
256     - Prior iteration: `../2026-06-human-gt-tuning/` · Study:
`../../research/OWNED_MODEL_TRAINING_STUDY.md`
257     - Code: `backend/eval/{eval_partial_labels,calibrate_local,rally_seq_proto}.py` ·
Checkpoints: `../../checkpoints/2026-06-eval-baseline/`
258
```

39. user (2026-06-10T05:37:41.633Z)

Byte-compiled / optimized / DLL files

```
__pycache__/
*.py[cod]
```

```
*$py.class
*.pt
```

Local environment secrets

```
.env
*.env
.env.*
```

C extensions

```
*.so
```

Distribution / packaging

```
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
share/python-wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST
```

SQLite databases & Local output files

```
*.db
*.db-journal
*.db-wal
output/
output_dir/
backend/static/highlights_*.mp4
backend/static/*_compiled.mp4
```

OS generated files

```
.DS_Store
Thumbs.db
desktop.ini
```

Coverage data

```
.coverage
.coverage.*
htmlcov/
.pytest_cache/
```

Local Claude Code config (machine-specific)

```
.claude/
.■c■o■v■e■r■a■g■e■
■
■---tracked-output-files-end---
```

40. user (2026-06-10T05:37:42.769Z)

```
10:    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
```

```
0.5):
12:        self.end_padding = end_padding
13:        self.begin_padding = begin_padding
67:        into a seamless highlights file, applying end_padding to prevent abrupt endings.
90:            logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
125:                padded_start = max(0.0, start - self.begin_padding)
126:                padded_end = end + self.end_padding
131:                    logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
```

41. user (2026-06-10T05:37:52.133Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\run_process_and_stitch.py:134:    compiler =
VideoCompiler(output_dir, end_padding=end_padding, begin_padding=begin_padding)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:201:    compiler =
VideoCompiler(output_dir)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:324:        compiler =
VideoCompiler(out_dir)
```

42. user (2026-06-10T05:37:53.093Z)

```
4d04a73 Quality iteration: human-GT multi-video LOVO + learned prototype + multi-court finding +
owned-model study/plan (#61)
90fb8b3 Human-GT partial-label eval harness + tuning + iteration doc (#60)
7b9dad2 fix(tests): cross-platform support for Windows-style paths in eval batch resolver and
tracknet/wasb basename logic
Python 3.13.13
```

43. user (2026-06-10T05:38:01.153Z)

```
175             "server": req.server,
176             "receiver": req.receiver,
177             "duration_min": req.duration_min,
178             "event_type": req.event_type
179         }
180         segments = db_instance.query_play_segments(req.video_id, filters)
181         return {"play_segments": segments}
182
183     @app.post("/api/compile")
184     def compile_highlights(req: CompileRequest):
185         video = db_instance.get_video(req.video_id)
186         if not video:
187             raise HTTPException(status_code=404, detail="Indexed video record not found.")
188
189         # Retrieve matching play segments from db
190         all_segments = db_instance.query_play_segments(req.video_id, {})
191         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
192
193         if not matched_segments:
194             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
195
196         # Extract start/end time pairs
197         time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
198
199         # Run compiler
200         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
201         compiler = VideoCompiler(output_dir)
202         try:
203             out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
204             return {"status": "success", "filepath": out_path}
```

```
205         except Exception as e:
206             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")
207
208     # --- CLI Debug Utility & Orchestration ---
209     def run_cli_diagnose_clip(video_path: str, timestamp: float):
210         """CLI debugging utility to examine segment properties around a timestamp."""
211         print("=" * 60)
212         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
213         print("=" * 60)
214
215         cfg = load_config()  # legacy wrapper still works, returns dict for now
216
217         # 1. Validation check diagnostics
218         print("[DIAGNOSTIC] Running validation framework...")
219         results = registry.run_all(video_path, cfg)
```

44. assistant (2026-06-10T05:38:10.634Z)

Now let me verify the test suite runs (offline/mocked, sanctioned) ? focusing on the eval tests.

45. user (2026-06-10T05:38:18.579Z)

```
.................................................................... [ 63%]
.......................................                              [100%]
113 passed in 3.64s
```

46. user (2026-06-10T05:38:30.781Z)

```
26:def test_save_and_load_baseline_roundtrip(tmp_path):
33:def test_load_baselines_absent_is_empty(tmp_path):
39:def test_no_baseline_is_not_a_regression(db, tmp_path):
46:def test_matching_baseline_passes(db, tmp_path):
53:def test_recall_drop_beyond_tolerance_regresses(tmp_path):
67:def test_within_tolerance_does_not_regress(tmp_path):
81:def test_regress_with_provider_file_tier(db, tmp_path):
93:def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
108:def test_cli_regression_exits_nonzero(tmp_path, capsys):
```

47. user (2026-06-10T05:38:31.459Z)

```
12:def test_iou_identical():
16:def test_iou_no_overlap():
21:def test_iou_partial():
26:def test_iou_contained():
31:def test_iou_degenerate_returns_zero():
38:def test_match_one_to_one():
47:def test_match_greedy_prefers_higher_iou():
57:def test_match_below_threshold_is_unmatched():
66:def test_match_one_gt_not_double_claimed():
76:def test_score_perfect():
88:def test_score_with_fp_and_fn():
101:def test_score_boundary_error():
110:def test_score_empty_predictions():
118:def test_score_empty_ground_truth():
124:def test_score_as_dict_roundtrip():
132:def test_pr_curve_default_thresholds():
138:def test_pr_curve_stricter_threshold_drops_loose_match():
```

48. assistant (2026-06-10T05:41:26.309Z)

All evidence gathered ? 113 eval-stack tests pass offline (3.64s), and I've verified every claim against source. Compiling the audit.

49. user (2026-06-10T05:44:01.878Z)

```
Structured output provided successfully
```

50. assistant (2026-06-10T05:44:02.574Z)

```
You've hit your session limit · resets 3:50pm (Asia/Calcutta)
```